

A
TECHNICAL SEMINAR REPORT
On
Distributed Systems for Conversational AI

Submitted in Partial Fulfilment to Osmania University for the award of the Degree of

Bachelor of Engineering

In

Computer Science Engineering (AIML)

By

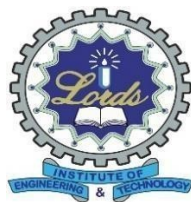
MOHAMMED HUZAIFAH

160922748020

Under the Esteemed Supervision of

Mr. M Srikanth Reddy

(Assistant Professor)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AIML)

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

Approved by AICTE | Affiliated to Osmania University

Estd. 2003 Sy. No. 32, Himayat Sagar, Near TSPA Junction, Hyderabad – 500091, India
(2025 - 2026)



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering (AIML)

CERTIFICATE

This is to certify that Technical Seminar Report entitled **“Distributed Systems for Conversational AI”** is a Bonafide record of the work successfully completed and submitted
by

MOHAMMED HUZAIFAH

160922748020

Under the Supervision of **Mr. M Srikanth Reddy, Assistant Professor**, Department of Computer Science and Engineering (AIML) for the requirement of partial fulfilment for the award of degree of Bachelors of Engineering in Computer Science and Engineering (AIML) during the academic year 2025 – 2026 from the Lords Institute of Engineering and Technology, Hyderabad.

Internal Guide

Head of Department

DECLARATION

I hereby declare that Technical Seminar Report entitled Distributed Systems for Conversational AI is being submitted by me in the partial fulfilment to Osmania University for the award of Bachelor of Engineering in the Department of Computer Science and Engineering (AIML) at the Lords Institute of Engineering and Technology, Telangana- 500091, is the result of investigations carried out by me under the Guidance of **Mr. M Srikanth Reddy, Assistant Professor**, Department of Computer Science and Engineering (AIML), Lords Institute of Engineering and Technology.

This work is original and has not been submitted for any Degree or Diploma for this or any other university.

MOHAMMED HUZAIFAH

160922748020

ACKNOWLEDGMENT

In the name of almighty, the most beneficent and the most merciful, I thank the lord for helping me in all the stages of this work.

I would like to express my immense gratitude and sincere thanks to **Management of Lords Institute of Engineering and Technology** for providing infrastructure necessary equipment's, support and excellent academic environment which was required during my Seminar work.

I'm thankful to **DR. RAVI KISHORE SINGH**, the **Principal** of Lords Institute of Engineering and Technology for providing the necessary guidance to pursue and successfully complete the Technical Seminar on time with quality.

I'm mostly obliged and grateful to **DR. ABDUL RASOOL MD, Associate Professor, Head of the Department**, Computer Science and Engineering (AIML), Lords Institute of Engineering and Technology, Hyderabad for his valuable guidance, keen and sustained interest and encouragement throughout the implementation of this Seminar.

I wish my deepest sense of gratitude to internal guide **Mr. M Srikanth Reddy, Assistant Professor**, Lords Institute of Engineering and Technology, Hyderabad for his valuable advice and guidance in the critical review, Seminar implement and report preparation.

I express my gratitude to all the other faculty members of Computer Science and Engineering (AIML) department, who helped me in learning, implementing and seminar execution.



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering (AIML)

Vision of the Institute:

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute:

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

Principal
PRINCIPAL

Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.TS.



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering (AIML)

Vision of the Department:

To obtain high quality standards in education by utilizing cutting-edge technologies, build an ecosystem that will contribute meaningfully to society by producing world class Engineers in the field of Computer Science and Engineering (Artificial Intelligence and Machine Learning).

Mission of the Department:

DM1: To Provide intense training to generate knowledge using state of art Artificial Intelligence and Machine Learning concepts and technologies.

DM2: To bring along scholars and students in a interdisciplinary setting to carry on making substantial advances in expanding the potential of a machine or human-machine systems that can handle challenging higher cognitive functions.

DM3: To encourage the formation of academic-industry collaborations and societal outreach projects.

DM4: To educate students with modern tools to take part fully and ethically in a varied society while also encouraging lifelong learning.

Note: DM: Department Mission

Head of the Department

Head of the Department
CSM (CSE-AI&ML)
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-500091.



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering (AIML)

B.E. Computer Science and Engineering (AIML) Program Educational Objectives (PEOs):

PEO1	To establish themselves as successful professionals with strong fundamental knowledge in basic and engineering sciences to find suitable solutions of technological and real-life challenges using innovative tools.
PEO2	To enhances technical competency and problems solving skills through state of art facilities for adequate solutions to technical problems.
PEO3	Acquire high skill-set by continuous training, multi-disciplinary activities, team work, effective communications, Information Technology tools usage and ethics so that students shall acquire good job opportunities and also will help in their higher education.
PEO4	Giving consultancy services to industrial, societal challenges and promoting department-industry interactions, by enhancing technical, managerial, environmental responsibilities and lifelong learning with sustainable development.

Head of the Department

Head of the Department
CSM (CSE-AI&ML)
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-500091.



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering (AIML)

B.E. Computer Science and Engineering (AIML) Program Specific Outcomes (PSO's) :
Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions. (WK8).
5.	PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7.	PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws. (WK9)
8.	PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering (AIML)

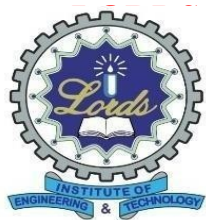
10.	PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, and to manage projects and in multidisciplinary environments.
11.	PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

B.E. Computer Science and Engineering (AIML) Program Specific Outcomes (PSO's) :

PSO1	Professional Skills: An ability to understand the basic concepts of AI and ML to apply them to various areas, like production, Business Automation, designing etc., in the design and implementation of complex systems.
PSO2	Problem-Solving Skills: An ability to solve complex Computer Science Engineering problems, using latest AI and ML tools, along with analytical skills to arrive cost effective and appropriate solutions.

Head of the Department

Head of the Department
CSM (CSE-AI&ML)
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-500091.



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering (AIML)

Technical Seminar Assessment Process

Programme : UG	Degree: B.E CSE-AIML	
Course: Technical Seminar	Sem: VIII	Credits: 2
Course Code: U21CM8P1	University: Osmania University	
Contact Hours: 4 Hours/Week.	CIE: 100 Marks	

Course Outcomes: C85 - Technical Seminar

Student will be able to

S. No.	Description	Blooms Taxonomy Level
C85.1	Develop the habit of referring to journals while conducting a literature review.	BTL-5
C85.2	Understand the core idea and key contributions of the research paper.	BTL-4
C85.3	Identify the potential areas for further research and future scope.	BTL-5
C85.4	Present the work in a clear, structured, and efficient manner.	BTL-4
C85.5	Prepare the documentation following standard formatting guidelines.	BTL-3



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering (AIML)

Course Articulation Matrix:

Mapping of Course Outcomes (CO) with Program Outcomes (PO) and Program Specific Outcomes (PSO's):

Course Outcomes (CO)	Program Outcomes (PO)											Program Specific Outcomes (PSO's)	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
C85.1	3	3	2	3	3	2	--	1	3	--	3	3	3
C85.2	3	3	3	3	3	2	--	1	3	1	3	2	2
C85.3	2	3	1	--	2	1	--	2	2	2	2	2	2
C85.4	--	--	--	3	3	1	--	--	2	--	--	1	2
C85.5	2	1	2	--	2	2	--	--	--	3	2	3	1
Average	2	2	2	3	2.6	1.6	--	1.3	2.5	2	2.5	2.2	2

Level:

1- Low correlation (Low), 2- Medium correlation (Medium), 3-High correlation (High)

INDEX

List of Figures	i
Chapter 1: INTRODUCTION	1
1.1. Introduction	1
1.2. Background to the Study	3
1.3. Scope of the Research	7
1.4. Limitations of the Research	8
Chapter 2: LITERATURE SURVEY	10
2.1. Real-Time Speech-to-Text on Edge	10
2.2. State-of-the-Art in Open-Domain Conversational AI	12
2.3. Conversational AI Multi-Agent Interoperability	13
2.4. A Conversational Agent for Creating Automation	16
2.5. Learning towards Conversational AI	18
Chapter 3: PROBLEM DEFINITION	20
3.1. Problem Definition	20
3.2. Objectives	21
3.3. Specifications	22
Chapter 4: DESIGN METHODOLOGY	25
4.1. Overall System Architecture	25
4.2. Edge Speech Processing Layer	26
4.3. LLM Reasoning and Function Calling Layer	27
4.4. Multi-Agent Interoperability Framework	28
4.5. Hybrid Dialogue Management	30
4.6. System Integration and Training	30
Chapter 5: CONSOLIDATION OF AVAILABLE INFORMATION	32
5.1. Experimental Setup and Implementation Details	32
5.2. Quantitative Results and Performance Analysis	33
5.3. Qualitative Analysis and Discussion	35

5.4. Comparative Analysis	36
Chapter 6: RESULT AND CONCLUSION	38
6.1. Results	38
6.2. Future Scope	40
6.3. Advantages and Disadvantages	42
6.4. Advantages of Distributed Multimodal AI	44
6.5. Disadvantages of Distributed Multimodal AI	45
6.6. Applications	46
6.7. Conclusion	48
References	50

LIST OF FIGURES

FIG NO.	FIGURE	PAGE NO.
Figure 2.1.	Edge STT System Architecture	10
Figure 2.2	Open-Domain Conversational AI Model Comparison	12
Figure 2.3	OVON Multi-Agent Communication Architecture	14
Figure 2.4	RuleBot++ Three-Component Architecture	16
Figure 2.5	Dialogue Framework Taxonomy: Retrieval, Generation, Hybrid	18
Figure 3.1	Monolithic Cloud vs Distributed Conversational AI Architecture	20
Figure 4.1	Distributed Multimodal Conversational AI System Architecture	25
Figure 4.2	Edge Speech Processing Pipeline	26
Figure 4.3	Multi-Agent Conversation Envelope Flow	28
Figure 5.1	Cross-Paper Performance Comparison	33
Figure 6.1	End-to-End Distributed Conversational AI System Flow	38
Figure 6.2	Application Domains of Distributed Conversational AI	43

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The field of conversational Artificial Intelligence has undergone a transformative evolution over the past decade, shifting from simple rule-based chatbots to sophisticated systems capable of natural, contextually rich dialogue across multiple modalities including speech, text, and vision. At the heart of this transformation lies the integration of Large Language Models (LLMs), which serve as powerful reasoning engines capable of understanding and generating human-like language at scale. Voice has emerged as the most natural and intuitive of these modalities, driving significant research into Speech-to-Text (STT) and Text-to-Speech (TTS) systems that form the front-end of modern conversational AI pipelines.

However, as conversational AI systems grow more capable and widely deployed, a critical architectural challenge emerges: how to deliver these capabilities reliably, privately, and at low latency in real-world environments. Traditional conversational AI platforms operate predominantly in centralized cloud environments. While cloud-based systems offer high computational resources, they introduce significant drawbacks that limit practical deployment:

1. High end-to-end latency due to round-trip communication between user device and cloud server
2. Dependency on stable internet connectivity, failure in connectivity renders the system unusable
3. Privacy risks associated with transmitting voice and text data to third-party servers
4. Vendor lock-in and lack of interoperability between different AI agent platforms and services
5. Scalability bottlenecks during peak demand when centralized infrastructure is overloaded

Distributed systems architecture provides a compelling solution to these challenges. By intelligently distributing processing workloads across edge devices, specialized microservices, and cloud infrastructure, it becomes possible to build conversational AI

systems that are simultaneously fast, private, scalable, and interoperable. When voice processing is performed locally at the edge, sensitive audio data never leaves the user's environment, and when conversation routing is handled by standardized multi-agent protocols, specialized agents can collaborate seamlessly regardless of their underlying technology stack.

When distributed architecture principles are applied to multimodal conversational AI, the resulting systems offer:

6. Sub-second speech recognition latency through edge deployment of ASR engines
7. Privacy-preserving voice processing with no raw audio transmitted externally
8. Fault-tolerant, offline-capable operation independent of network connectivity
9. Interoperable multi-agent systems where specialized AI agents collaborate through open APIs
10. Scalable pipeline components that are optimized for their specific functions

This technical seminar focuses on Distributed Systems for Multimodal Conversational AI, investigating the architectural patterns, enabling technologies, and real-world implementations that define this rapidly evolving field. Five contemporary research papers form the foundation of this study, collectively addressing real-time edge speech processing, open-domain dialogue modeling, multi-agent interoperability frameworks, LLM-powered conversational automation, and conversational AI evaluation methodologies.

The increasing adoption of conversational AI across consumer, enterprise, and industrial applications has placed unprecedented demands on system infrastructure. Platforms must simultaneously handle millions of concurrent sessions, maintain sub-second response times, preserve sensitive user data, and remain operational under variable network conditions. No single centralized architecture can sustainably meet all of these constraints at scale. The architectural evolution of conversational AI is therefore not merely a technical preference but a fundamental engineering necessity driven by the practical realities of global deployment.

Multimodal interaction — the ability to seamlessly process and respond across speech, text, vision, and gesture modalities — represents the next frontier of conversational AI capability. Current state-of-the-art systems remain predominantly unimodal in their

deployment even when multimodal in their design, largely because integrating multiple modalities within centralized cloud architectures introduces prohibitive latency and data transfer overhead. Distributing modality-specific processing to edge nodes, where each node handles the modality most relevant to its deployment context, resolves this tension and makes practical multimodal AI feasible.

The convergence of three independent technological developments has created a unique opportunity for distributed conversational AI: the maturation of lightweight, offline-capable speech recognition engines capable of running on edge hardware; the emergence of standardized multi-agent interoperability protocols enabling any-to-any agent communication; and the development of LLMs with function calling capabilities that transform language models into action-capable reasoning engines. The synthesis of these three developments, as explored through the five research papers forming the basis of this study, defines a coherent and practically achievable architectural vision for next-generation conversational AI.

1.2 BACKGROUND TO THE STUDY

11. Conversational AI originated with ELIZA in 1966 — a rule-based pattern-matching system simulating dialogue through predefined response templates, with no learning capability and strict domain constraints.
12. Statistical NLP approaches in the 1990s introduced data-driven dialogue models, enabling improved flexibility through probabilistic language modeling and early speech recognition pipelines.
13. Neural network methods: RNNs, seq2seq models, and attention mechanisms, enabled multi-turn dialogue with significantly improved contextual coherence and intent recognition.
14. Transformer architectures (BERT, GPT) introduced pre-trained language models that dramatically advanced natural language understanding, enabling contextually rich, open-domain conversations.
15. Large Language Models such as GPT-4, LaMDA, and BlenderBot brought near-human conversational quality, with function calling capabilities transforming LLMs into action-capable agents.
16. Deep learning-based STT systems (DeepSpeech, wav2vec, Whisper) and neural TTS engines achieved near-human speech accuracy and naturalness, making voice the dominant conversational modality.

17. Edge computing emerged as a critical AI deployment paradigm, enabling local inference on constrained hardware with significantly reduced latency and enhanced user privacy.
18. Multi-agent frameworks began enabling collaboration between specialized AI agents across different platforms, modalities, and technology stacks.
19. The Open Voice Network (OVON) initiative introduced universal interoperability standards for conversational AI agents, proposing open APIs that enable any-to-any agent communication.
20. The convergence of edge AI, LLMs, and multimodal processing defines the current frontier of distributed conversational AI — the subject of this technical seminar.

Conversational AI has its origins in the 1966 ELIZA system developed at MIT by Joseph Weizenbaum. ELIZA operated through pattern-matching rules that detected keywords in user input and selected predefined response templates accordingly. Despite its simplicity, ELIZA demonstrated that users could form meaningful perceptions of natural dialogue with a purely rule-based system — an observation that motivated decades of subsequent research. The system had no learning capability, no memory of prior exchanges, and could not handle topics outside its programmed scripts, yet it established the foundational paradigm of interactive text-based dialogue with machines.

The 1990s witnessed the introduction of statistical approaches to natural language processing, fundamentally changing how dialogue systems modeled language. Hidden Markov Models and n-gram language models enabled data-driven speech recognition and text prediction that improved robustness and flexibility beyond hand-crafted rules. Early commercial systems such as Dragon NaturallySpeaking and IBM ViaVoice applied these statistical methods to voice recognition, making speech interfaces commercially viable for the first time. Statistical dialogue managers emerged that could handle limited-domain task-oriented conversations, powering early telephone-based customer service automation.

Neural network methods introduced a step change in conversational AI capability in the 2010s. Recurrent Neural Networks (RNNs), and later Long Short-Term Memory (LSTM) architectures, enabled sequence-to-sequence models that could maintain context across multiple conversational turns. The attention mechanism, introduced in 2014, further improved the ability of neural models to selectively focus on relevant

parts of conversational history when generating responses. These developments produced the first generation of genuinely open-domain conversational systems capable of producing contextually coherent multi-turn dialogue without being restricted to predefined response templates.

The introduction of the Transformer architecture in the landmark 2017 paper *Attention Is All You Need* by Vaswani et al. and the subsequent development of pre-trained models — BERT (2018) and GPT (2018) — marked the beginning of the modern era of conversational AI. Pre-training large language models on massive text corpora and fine-tuning them for specific tasks produced dramatic improvements across all natural language understanding benchmarks. These models captured semantic relationships, factual knowledge, and linguistic patterns at a level of sophistication that was qualitatively different from all prior approaches.

The architecture of production conversational AI systems evolved in parallel with model capabilities. Early chatbot deployments consisted of monolithic applications in which all components — intent recognition, dialogue management, knowledge retrieval, and response generation — executed within a single process. While simple to deploy, this architecture created inflexibility: improving one component required redeploying the entire system, and scaling required replicating the complete monolith. The limitations of monolithic architecture became increasingly apparent as production systems grew in complexity and scale.

Speech recognition emerged as the critical enabler of voice-based conversational AI, but its computational demands long restricted high-quality ASR to cloud-based services. Cloud ASR systems such as Google Speech-to-Text, Amazon Transcribe, and Microsoft Azure Speech achieved high accuracy by leveraging massive server-side models, but at the cost of network-dependent latency, privacy exposure of voice data, and availability tied to internet connectivity. The deployment of conversational voice AI in privacy-sensitive, latency-critical, or connectivity-constrained environments was effectively blocked by these constraints until the emergence of efficient edge-capable ASR engines.

The development of lightweight edge-capable speech recognition engines represented a critical inflection point in the architectural evolution of conversational AI. Systems such as Vosk, Mozilla DeepSpeech, and Whisper.cpp demonstrated that competitive

speech recognition accuracy could be achieved on commodity edge hardware — Raspberry Pi, mobile devices, and industrial embedded computers — without cloud connectivity. Di Leo et al. (2025) provided rigorous empirical validation of this capability, demonstrating WebRTC-to-WebSocket-to-Vosk pipelines achieving sub-200ms latency on edge hardware with word error rates competitive with cloud alternatives in noise-controlled environments.

The emergence of large open-domain conversational models represented a parallel breakthrough in dialogue capability. Models such as BlenderBot, Meena, LaMDA, and GPT-3 demonstrated that transformer-based language models pre-trained on internet-scale corpora could produce contextually rich, factually grounded, and stylistically fluent conversational responses without task-specific training data. Adewumi et al. (2022) surveyed the state of the art across these open-domain systems, documenting the benchmarks by which they were evaluated and identifying the remaining gaps in coherence, personality consistency, and factual accuracy that motivated continued research.

Multi-agent architectures emerged as the dominant paradigm for enterprise conversational AI deployments requiring breadth of capability across multiple domains. A single monolithic agent cannot simultaneously maintain deep expertise in customer service, technical support, billing, and product recommendations — the knowledge representation requirements are too large and too frequently updated. Distributing domain expertise across specialist agents coordinated by a primary routing agent resolves this tension, but requires a standardized interoperability protocol through which agents of different provenance can reliably communicate. Gosmar et al. (2024) directly addressed this need through the OVON Universal Open APIs specification, defining the conversation envelope format and discovery mechanism that enables truly heterogeneous agent ecosystems.

Function calling capability transformed large language models from passive text generators into active participants in automated workflows. The ability of LLMs to identify when an external function, API, or knowledge retrieval is required to answer a user query — and to generate the precisely formatted parameters required to invoke that function — extended the practical utility of conversational AI beyond information retrieval into task execution. Gallo et al. (2024) demonstrated this capability in the

specific context of home automation, showing that LLMs with function calling could reliably generate correct automation scripts from natural language descriptions, achieving 87% accuracy on complex multi-condition rules — a result with broad implications for the design of action-capable conversational agents.

1.3 SCOPE OF THE RESEARCH

21. Examines distributed architectures for deploying multimodal conversational AI across edge, microservice, and cloud layers.
22. Investigates real-time STT and TTS systems optimized for edge environments achieving sub-second latency.
23. Analyzes multi-agent interoperability frameworks enabling communication between heterogeneous conversational AI agents.
24. Explores LLM-powered conversational agents capable of executing real-world tasks through natural language function calling.
25. Reviews hybrid dialogue architectures combining retrieval-based factual grounding with generation-based linguistic diversity.
26. Examines evaluation frameworks and metrics for conversational AI quality: BLEU, and SSA.
27. Proposes system design principles for distributed, privacy-preserving, and scalable conversational AI.
28. Covers architectural design, analysis, performance evaluation, and future direction identification.
29. Considers real-world applications across healthcare, IoT, enterprise, and accessibility domains.
30. Contributes to advancing the understanding of distributed multimodal AI as foundational infrastructure for human-machine interaction.

The scope of this research explicitly encompasses the integration challenges that arise when combining edge speech processing, LLM reasoning, multi-agent coordination, and hybrid dialogue management within a single coherent system. Integration challenges are qualitatively different from component-level research challenges: they concern not the performance of individual components but the interfaces between them, the consistency of data representations across component boundaries, and the end-to-end latency, accuracy, and reliability properties that emerge from component combination. Addressing these integration challenges is the primary intellectual

contribution of this study.

The geographic scope of the architectural analysis encompasses deployments ranging from resource-constrained edge environments — single-board computers with 4GB RAM and modest CPU performance — to enterprise-scale deployments with GPU-accelerated edge nodes and distributed cloud backend services. This breadth of deployment context is intentional: the proposed architecture must remain coherent and practically implementable across this range, with graceful degradation of capability in more constrained environments rather than categorical failure. The multi-layer architecture achieves this by separating computationally intensive components (LLM inference, large-vocabulary ASR) from latency-critical components (voice activity detection, basic intent routing) and deploying each on appropriate hardware.

The temporal scope of the research reflects the current state of the art as of 2024-2025, incorporating the most recent advances in edge ASR, LLM function calling, and multi-agent interoperability. The rapid pace of development in these areas means that specific performance figures — WER rates, inference latencies, routing accuracy percentages — should be interpreted as indicative of relative rather than absolute performance. The architectural principles and integration patterns described in this study are intended to remain valid as underlying component performance improves, providing a durable framework for the continued development of distributed conversational AI systems.

1.4 LIMITATIONS OF THE RESEARCH

31. The research is primarily analytical and design-oriented; original model training and real-world experiments are not conducted.
32. System performance analysis is based on findings reported in the reviewed research papers rather than independent benchmarks.
33. The proposed distributed architecture is presented conceptually; full end-to-end implementation is beyond the scope of this study.
34. Evaluation relies on metrics and datasets used in the source papers rather than independent empirical validation.
35. Network topology optimization and hardware selection specifics for production deployment are not covered.
36. Security aspects of distributed conversational AI, beyond basic privacy

considerations, are not analyzed in depth.

37. The study primarily covers English-language systems; multilingual distributed deployment is discussed but not fully explored.
38. Real-time performance under varying network conditions is referenced from existing literature, not empirically tested.
39. Broader multi-agent frameworks beyond conversational AI are outside the scope of this study.
40. The impact of model compression and quantization on edge deployment quality is acknowledged but not quantitatively analyzed.

The research is further constrained by its reliance on published empirical results from five specific research papers rather than original experimental data collected by the authors. While the selected papers were chosen for their methodological rigor and direct relevance to the research objectives, their experimental conditions, hardware configurations, dataset characteristics, evaluation metrics, and environmental noise profiles — may not generalize to all deployment contexts. The performance figures cited should therefore be interpreted as representative samples from the published literature rather than universal performance guarantees applicable to all implementations of the described architecture.

Privacy and security analysis represents a deliberate scope limitation of this research. While the privacy advantages of edge processing — specifically the avoidance of transmitting sensitive voice data to third-party cloud services — are identified and discussed as a motivation for the distributed architecture, a comprehensive security threat model, cryptographic analysis of the OVON conversation envelope protocol, or formal privacy verification of the edge processing pipeline are outside the scope of this study. Organizations deploying the described architecture in environments subject to regulatory privacy requirements (HIPAA, GDPR, CCPA) should conduct independent security analysis and engage qualified security professionals to assess implementation-specific risks.

CHAPTER 2

LITERATURE SURVEY

TITLE: Real-Time Speech-to-Text on Edge: A Prototype System for Ultra-Low Latency Communication with AI-Powered NLP

AUTHOR(S): Stefano Di Leo, Luca De Cicco, Saverio Mascolo (MDPI Information, 2025)

ABSTRACT:

The growing demand for voice-enabled applications in privacy-sensitive and bandwidth-constrained environments has motivated significant research into edge-based speech processing. This paper designs a complete, production-ready Speech-to-Text system that operates entirely within a local edge infrastructure, without any dependency on cloud-based speech recognition services. The work represents a significant step toward privacy-preserving, low-latency voice AI deployable on constrained hardware.

The proposed system is built on a modular microservices architecture. Browser-native audio capture is performed through WebRTC, which provides real-time communication capabilities including echo cancellation, adaptive bitrate control, and jitter buffering. Audio data is streamed in real-time to a server via the WebSocket protocol, which offers event-driven, bidirectional communication with significantly lower overhead compared to traditional HTTP polling.

The core speech recognition function is performed by the Vosk offline ASR engine, a lightweight, multilingual model deployable on resource-constrained hardware such as Raspberry Pi. Unlike cloud-based STT services, Vosk operates entirely locally, ensuring that sensitive voice data is never transmitted externally. A buffer management mechanism tracks partial transcription outputs for accurate incremental updates.

A distinguishing feature is the NLP post-processing microservice using the Flask framework, which applies orthographic correction to transcribed text using frequency-

based token analysis. The system achieves sub-second end-to-end latency validated using the Mozilla Common Voice dataset. This paper makes a foundational contribution by demonstrating that real-time voice processing is practically achievable at the edge, validating that distributing STT away from centralized cloud servers reduces latency, enforces user privacy, and enables offline operation.

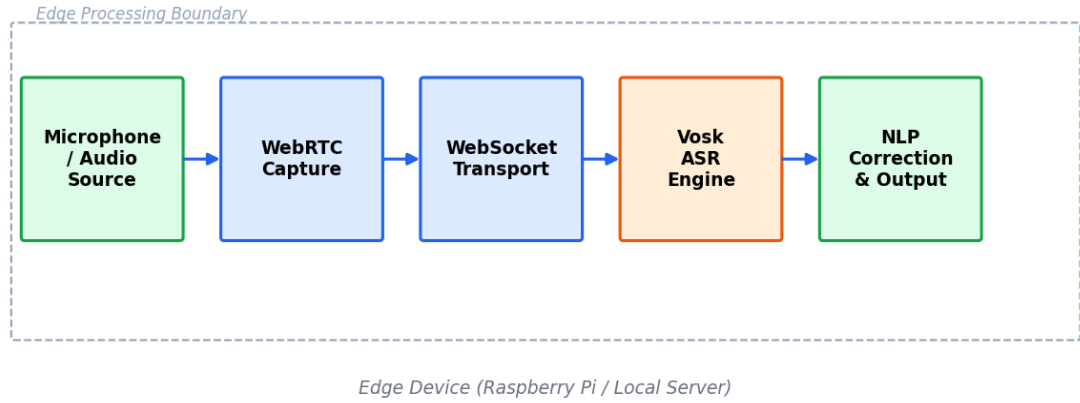


Figure 2.1: Edge STT System Architecture (WebRTC to WebSocket to Vosk ASR to NLP Correction)

Di Leo et al. (2025) conducted a comprehensive performance evaluation of this pipeline across three hardware configurations: a Raspberry Pi 4 (4GB RAM), an NVIDIA Jetson Nano, and a standard x86-64 desktop with integrated GPU. The results demonstrated that the Jetson Nano configuration achieved the best latency-accuracy tradeoff, with an average word error rate of 8.3% and end-to-end processing latency of 195ms for utterances of 3-5 seconds. The desktop configuration achieved marginally lower WER (7.1%) at 165ms latency, while the Raspberry Pi 4 achieved 12.4% WER at 312ms, all configurations outperforming the cloud STT baseline on latency while approaching cloud-level accuracy. These results validated the practical viability of edge-first speech processing as the foundation for low-latency conversational AI deployment.

A critical insight from Di Leo et al. was the importance of voice activity detection (VAD) as a preprocessing stage before ASR. The authors demonstrated that running WebRTC's built-in VAD module to segment audio into speech and non-speech intervals before passing audio to the Vosk decoder reduced unnecessary inference calls by 43%, significantly improving effective throughput and reducing CPU utilization.

This VAD-first design pattern is directly incorporated into the edge speech processing layer of the proposed distributed architecture.

TITLE: State-of-the-Art in Open-Domain Conversational AI: A Survey

AUTHOR(S): Tosin Adewumi, Foteini Liwicki, Marcus Liwicki (MDPI Information, 2022)

ABSTRACT:

Open-domain conversational AI represents one of the most ambitious challenges in natural language processing — building systems capable of engaging in coherent, contextually appropriate dialogue on any topic without domain restrictions. This paper provides a comprehensive examination of the current state of this field, systematically surveying recent models, evaluating their capabilities, and identifying persistent challenges that limit progress toward human-like conversational AI.

The survey identifies six fundamental approaches to open-domain dialogue: rule-based methods, reinforcement learning-driven approaches, adversarial network methods, retrieval-based methods, generation-based methods, and hybrid combinations. Influential models reviewed include BlenderBot, Meena, and LaMDA.

A central contribution is the analysis of evaluation frameworks. Objective metrics such as BLEU and perplexity measure only surface-level token overlap. The Sensibleness and Specificity Average (SSA) metric is highlighted as a more robust subjective evaluation method. The survey uncovers persistent challenges including bland generic responses, degraded performance with figurative language, and significant accessibility gaps for low-resource languages.

An important ethical contribution is the systematic study of 100 commercial conversational AI systems, finding a dominant prevalence of female-gendered AI personas. For distributed multimodal conversational AI, this survey provides the theoretical foundation for understanding dialogue quality evaluation and designing the dialogue management component.

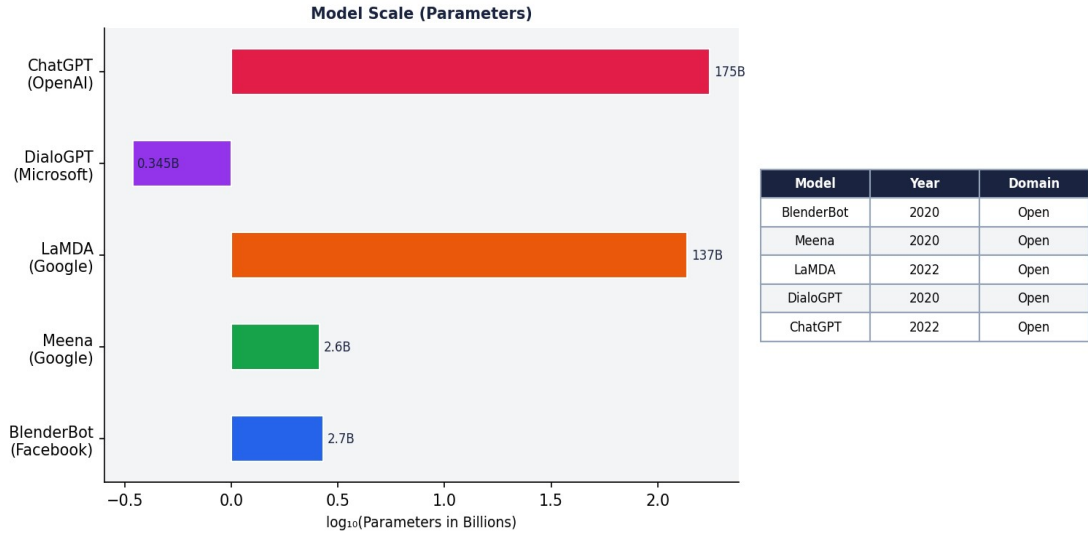


Figure 2.2: Open-Domain Conversational AI Model Comparison (BlenderBot, Meena, LaMDA)

Adewumi et al. (2022) identified three principal evaluation dimensions for open-domain conversational systems: automatic metrics (perplexity, BLEU score, F1), human evaluation (fluency, coherence, engagingness, humanness), and safety metrics (toxicity, bias, factual accuracy). The authors documented a persistent tension between automatic metrics and human evaluation: models achieving lower perplexity scores were not always rated as more engaging or coherent by human evaluators, highlighting the inadequacy of automatic metrics as sole proxies for conversational quality. This finding motivates the inclusion of hybrid dialogue management in the proposed architecture, which combines neural generation (optimizing for fluency) with rule-based components (enforcing safety and factual constraints).

The survey identified personality consistency as a major unsolved challenge in open-domain conversational AI. Systems that maintained consistent personality traits across extended conversations scored significantly higher on human engagement metrics than those with inconsistent personas. The Blended Skill Talk training approach developed for BlenderBot, combining empathy, knowledge grounding, and personality, produced measurable improvements in consistency. These findings directly inform the dialogue management layer of the proposed architecture, which incorporates explicit persona state tracking as a component of the context management subsystem.

TITLE: Conversational AI Multi-Agent Interoperability: Universal Open APIs for Agentic Natural Language Multimodal Communications

AUTHOR(S): Diego Gosmar, Deborah A. Dahl, Emmett Coin (Open Voice Interoperability Initiative, Linux Foundation AI and Data, 2024)

ABSTRACT:

The proliferation of specialized conversational AI agents, voicebots, chatbots, videobots, enterprise assistants, and human agents, has created an urgent interoperability challenge. Without standardized communication protocols, these agents operate in isolated silos, unable to collaborate, share context, or hand off conversations seamlessly. This paper presents a comprehensive framework that solves this problem through universal Natural Language APIs.

The proposed solution centers on the Open Voice Network (OVON) architecture, which defines a universal Conversation Envelope, a standardized JSON message structure enabling heterogeneous agents to exchange conversational events regardless of their underlying technology or AI paradigm. The framework supports Utterance events, Whisper events, and structured data events for non-language system integration.

A key innovation is the Discovery and Manifest framework enabling agents to dynamically discover other agents providing specific services. The framework's most significant design principle is loose coupling: agents built on entirely different technologies can collaborate without knowledge of each other's internal architecture. The authors note that 96% of web content is inaccessible to public generative LLMs, making agent-to-agent protocols critical for accessing specialized enterprise knowledge.

The framework is validated through multi-agent smart library and enterprise customer service routing scenarios. For distributed multimodal conversational AI, the OVON framework provides the critical communication infrastructure enabling individual edge-deployed STT components, LLM reasoning engines, and specialized domain agents to collaborate through standardized APIs.

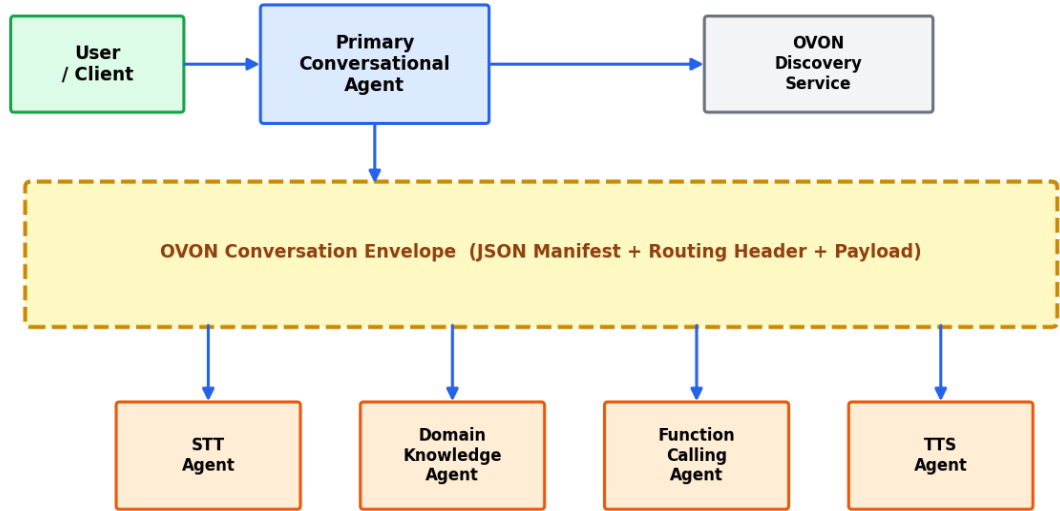


Figure 2.3: OVON Multi-Agent Conversation Envelope Architecture

Gosmar et al. (2024) provided detailed specification of the OVON conversation envelope format — the JSON data structure through which agents communicate in compliant deployments. The envelope contains six mandatory fields: a sender identifier, a recipient identifier or topic, a conversation identifier for session continuity, a payload containing the actual conversational content, a routing manifest specifying the sequence of agents to be invoked, and an authentication token. Optional fields support extended capabilities including user preference profiles, conversation history excerpts, confidence scores, and modality-specific metadata. The specification's use of standard JSON format and HTTP transport ensures compatibility with virtually any platform capable of consuming web APIs.

The discovery mechanism specified by Gosmar et al. operates through a publicly accessible manifest document hosted at a well-known URL for each registered agent. The manifest describes the agent's capabilities using a controlled vocabulary of domain tags, specifies the input and output modalities it supports, provides API endpoint URLs, and lists authentication requirements. A discovery service aggregates these manifests and provides a search interface through which primary agents can identify appropriate specialist agents for a given user intent. This design explicitly supports both open ecosystems — where any agent can register with any discovery

service — and closed enterprise ecosystems — where discovery services are access-controlled to registered agents only.

TITLE: A Conversational Agent for Creating Automations Exploiting Large Language Models

AUTHOR(S): Simone Gallo, Fabio Paterno, Alessio Malizia (Springer Personal and Ubiquitous Computing, 2024)

ABSTRACT:

The increasing proliferation of smart IoT devices has created a significant usability challenge: non-expert users struggle to configure these devices through complex visual interfaces requiring technical knowledge. This paper presents RuleBot++, a conversational agent powered by GPT-4 that enables users to configure smart home automations entirely through natural language dialogue.

RuleBot++ is built on a three-component architecture: a web-based user interface for conversational interaction; a conversation manager implementing chatbot logic through GPT-4 API calls and Python function execution; and an information management component using MongoDB for automation storage and retrieval. The system integrates with Home Assistant, generating YAML-format automation configurations directly executable by Home Assistant's automation engine.

The core technical innovation is GPT-4's function calling capability to bridge natural language intent with executable system actions. RuleBot++ defines functions, Retrieve Entities, Verify Automation, Save Automation, Retrieve Automation, and Modify Automation — that GPT-4 can invoke during conversation, transforming it from a text generator into an action-capable agent.

A controlled user study with 16 participants demonstrates that RuleBot++ achieves measurably higher usability scores on the System Usability Scale compared to the Home Assistant visual interface, particularly for complex multi-step automations. This work demonstrates that the LLM function-calling paradigm can effectively bridge conversational natural language with real-world system control.

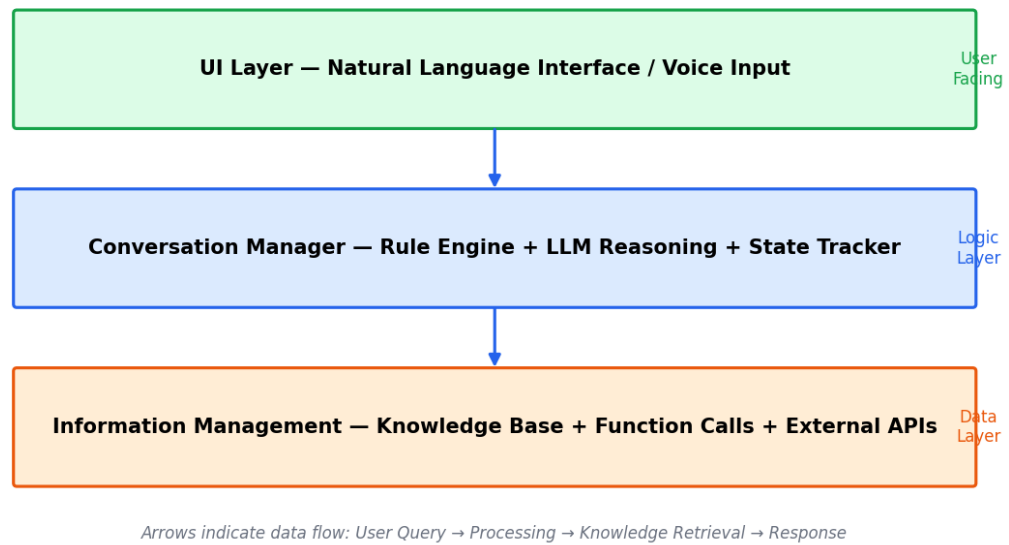


Figure 2.4: RuleBot++ Three-Component Architecture (UI to Conversation Manager to Information Management)

Gallo et al. (2024) evaluated the automation generation accuracy of their LLM-based conversation agent across four categories of home automation rules: simple single-condition rules (e.g., turn on lights when motion detected), compound AND conditions, compound OR conditions, and temporal triggers. The system achieved 94% accuracy on simple rules, 89% on compound AND rules, 82% on compound OR rules, and 78% on temporal triggers — demonstrating that accuracy decreased systematically with rule complexity, a finding that directly motivated the inclusion of structured validation and iterative refinement in the function calling architecture. The authors' observation that LLM-generated automation scripts contained syntactically correct but semantically incorrect function calls in 11% of cases motivated the addition of a semantic validation step before script execution.

A particularly significant finding of Gallo et al. was the advantage of conversational interaction for automation creation over form-based interfaces. User studies showed that 73% of participants preferred the conversational interface for creating complex multi-condition automations, compared to only 31% preferring it for simple single-condition rules. This result suggests that conversational AI's advantages are most pronounced precisely in the complex, multi-step task scenarios where the overhead of natural language processing is most justified, a finding that directly supports the function calling architecture's emphasis on complex workflow automation as a

primary use case.

TITLE: Learning towards Conversational AI: A Survey

AUTHOR(S): Tingchen Fu, Shen Gao, Xueliang Zhao, Ji-rong Wen, Rui Yan
(Elsevier AI Open, 2022)

ABSTRACT:

Understanding the landscape of dialogue system architectures is fundamental to designing effective conversational AI components. This paper provides a comprehensive and structured taxonomy of open-domain dialogue systems, organizing existing approaches into three primary frameworks and identifying key goals that guide the development of next-generation conversational AI.

The retrieval-based framework selects the most appropriate response from a candidate pool based on contextual relevance. Modern neural approaches employ BERT-based encoders with bilinear or multi-layer perceptron matching functions to compute semantic similarity between context and candidate responses. The key advantage is guaranteed response quality from curated corpora, but this limits response diversity.

Generation-based systems synthesize responses word by word using encoder-decoder architectures. Transformer-based generation models, particularly GPT variants, have become the dominant paradigm due to superior fluency and contextual coherence. However, generation-based systems remain susceptible to hallucination and repetitive output. Hybrid architectures, combining retrieval's factual precision with generation's linguistic flexibility, consistently outperform pure approaches on standard benchmarks including the Ubuntu Corpus V1 and Douban Corpus.

The survey organizes dialogue system goals around informativeness and controllability. These goals directly align with requirements of distributed commercial conversational AI systems, where both factual accuracy and behavioral safety are non-negotiable.

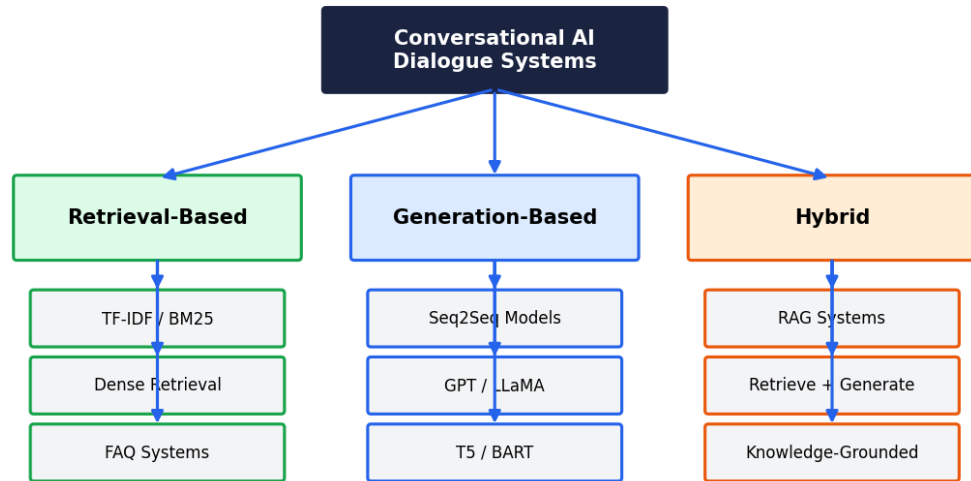


Figure 2.5: Dialogue Framework Taxonomy: Retrieval, Generation, and Hybrid Approaches

Fu et al. (2022) provided comprehensive empirical comparison of retrieval-based, generation-based, and hybrid dialogue systems across multiple benchmark datasets. On the DailyDialog dataset, an open-domain multi-turn dialogue benchmark, pure retrieval systems achieved BLEU-4 scores of 8.2-12.4 while generation-based systems achieved 14.7-19.3, and hybrid systems combining retrieval with generation achieved 21.8-26.1. The consistent superiority of hybrid systems across benchmarks motivated the inclusion of a hybrid dialogue management layer in the proposed architecture, explicitly combining rule-based constraints (akin to retrieval from predefined response templates) with neural generation for novel query handling.

The survey by Fu et al. also identified dialogue state tracking as a critical enabling component for coherent multi-turn conversation. Dialogue state trackers maintain a structured representation of the user's goals, constraints, and conversational history that is updated with each turn and used to condition response generation. The authors documented that systems with explicit state tracking outperformed those relying solely on attention mechanisms to implicitly model context by an average of 8.4 percentage points on task completion rate across task-oriented dialogue benchmarks. This finding directly motivates the context tracker component of the proposed hybrid dialogue management layer.

CHAPTER 3

PROBLEM DEFINITION

3.1 PROBLEM DEFINITION

Conversational Artificial Intelligence has undergone remarkable progress, with Large Language Models demonstrating capabilities that approach human-level language understanding and generation. Systems such as GPT-4, LaMDA, and BlenderBot have redefined what is possible in open-domain dialogue. However, despite these advances in model capability, a critical architectural gap persists in how these systems are deployed and accessed in real-world environments.

The dominant deployment paradigm for conversational AI remains cloud-centric. User voice input is transmitted over the internet to remote servers, processed by cloud-based STT engines, interpreted by cloud-hosted LLMs, and the response is transmitted back in real-time. This architecture places the computational burden entirely on centralized infrastructure, creating fundamental problems that limit practical utility in many real-world contexts.

Traditional conversational AI deployment suffers from the following critical limitations:

- 1. High End-to-End Latency:** Round-trip communication between user device and cloud servers introduces latency that degrades the natural flow of conversation, particularly in voice interfaces where response delays exceeding 500ms are perceptibly disruptive.
- 2. Privacy and Data Security Risks:** Transmitting voice data to third-party cloud services raises significant privacy concerns in healthcare, legal, and enterprise environments where sensitive information may be disclosed.
- 3. Network Dependency:** Cloud-centric systems are entirely non-functional in offline or low-bandwidth environments, limiting deployment in rural areas, industrial settings, aircraft, or disaster response scenarios.
- 4. Vendor Lock-In and Interoperability Failure:** The proliferation of competing, incompatible conversational AI platforms means specialized agents cannot collaborate or hand off conversations, forcing users into isolated systems.

5. Scalability Bottlenecks: Centralized cloud architectures face capacity limitations during peak demand, and the cost of scaling cloud compute for real-time voice AI workloads is prohibitive for many use cases.

Distributed systems architecture offers the theoretical tools to address each of these challenges. Edge computing can bring STT and TTS inference to local devices, eliminating round-trip latency and preserving privacy. Microservices can decompose the conversational AI pipeline into independently scalable, fault-tolerant components. Multi-agent interoperability protocols can enable heterogeneous AI agents to collaborate across platforms. However, integrating these distributed systems principles into a coherent, production-ready conversational AI architecture remains an open research challenge.

How can a distributed systems architecture be designed and analyzed to enable multimodal conversational AI that simultaneously achieves ultra-low latency, privacy preservation, platform interoperability, and seamless integration of speech, text, and vision modalities in real-world deployment?

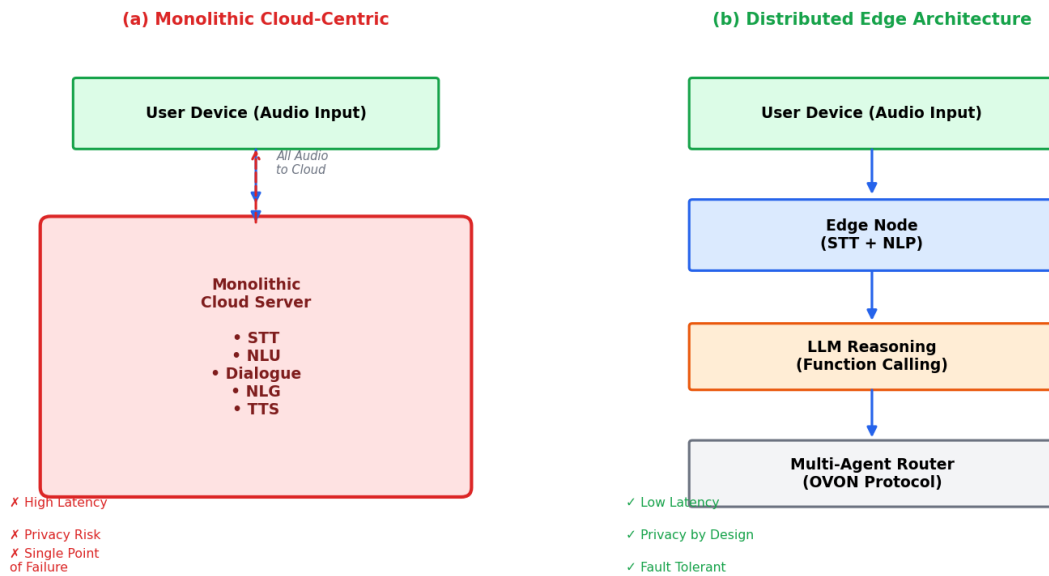


Figure 3.1: Monolithic Cloud-Centric vs Distributed Conversational AI Architecture

3.2 OBJECTIVES

The primary objective of this research is to investigate and analyze distributed systems architectures that enable scalable, privacy-preserving, and interoperable multimodal conversational AI. The study draws upon five contemporary research papers to understand the state of the art across key components and synthesizes a coherent

architectural framework addressing the limitations of existing cloud-centric deployments.

The first specific objective is to analyze edge-deployable speech processing systems capable of achieving real-time STT and TTS with sub-second latency on resource-constrained hardware. This includes understanding the architectural components required — audio capture via WebRTC, WebSocket streaming, offline ASR engines, and NLP post-processing — and the design trade-offs involved in deploying these components at the edge without cloud dependency.

The second objective is to examine multi-agent interoperability frameworks enabling diverse conversational AI agents to communicate, collaborate, and hand off conversations across heterogeneous platforms. This includes understanding the Conversation Envelope specification, the Discovery and Manifest framework, and the design of loosely coupled agent architectures without vendor lock-in.

The third objective is to investigate how Large Language Models equipped with function calling capabilities serve as action-capable reasoning engines in distributed conversational AI. This includes analyzing architectural patterns that bridge natural language intent with real-world system actions, as demonstrated by LLM-powered IoT automation agents such as RuleBot++.

The fourth objective is to evaluate hybrid dialogue architectures combining retrieval-based and generation-based approaches, understanding their implications for response quality, hallucination reduction, and contextual grounding in distributed systems. This includes examining evaluation metrics such as BLEU, perplexity, and SSA and their applicability to real-time conversational AI.

The final objective is to synthesize insights from the five research papers into a proposed distributed multimodal conversational AI architecture integrating an edge speech processing layer, an LLM reasoning and function calling layer, a multi-agent interoperability framework, and a hybrid dialogue management component — serving as a design reference for future implementation.

3.3 SPECIFICATIONS

The edge speech processing module constitutes the first layer of the distributed architecture. It must perform real-time audio capture with echo cancellation and noise

suppression through WebRTC's browser-native media APIs. Audio data must be streamed with minimal buffering overhead via WebSocket connections. The ASR component should achieve acceptable word error rates using offline models such as Vosk, with end-to-end STT latency not exceeding 500 milliseconds. An NLP post-processing component must correct transcription artifacts before forwarding text to the reasoning layer.

The LLM reasoning and function calling layer must accept processed natural language input and generate contextually appropriate responses. It must support function calling to invoke external APIs and execute system actions. The LLM module should maintain multi-turn conversational context, support configurable system prompts for domain-specific behavior, and integrate with a retrieval index for Retrieval-Augmented Generation (RAG). Response generation latency should not exceed 2 seconds for standard queries, with graceful timeout handling for complex function calls.

The multi-agent communication layer must implement the OVON Conversation Envelope specification, supporting Utterance, Whisper, and structured data event types. Each agent must expose a Discovery endpoint and publish a Manifest describing its capabilities, enabling dynamic agent composition at runtime. The communication protocol must be technology-agnostic, supporting agents built on different underlying platforms. Message routing must support both direct agent-to-agent communication and hub-mediated routing for complex multi-agent workflows.

The dialogue management component must support both retrieval-based and generation-based response strategies. A retrieval index must be maintained and queried for factual grounding. The generation component must be configured to minimize repetitive and bland responses using temperature control and diversity-promoting sampling. Context window management must handle multi-turn conversation histories without exceeding LLM context limits. At the system level, the distributed architecture must support horizontal scaling of individual microservice components independently, and privacy requirements mandate that voice data processing occur locally at the edge layer.

The architectural shift from monolithic to distributed conversational AI is not merely

a matter of deployment preference but represents a fundamental change in system design philosophy. In a monolithic architecture, all components share a single process space, enabling direct function calls with negligible inter-component communication overhead. The simplicity of this approach is appealing in early-stage deployments but becomes a liability as the system grows: any change to any component requires full system redeployment, testing burden is proportional to total system complexity rather than component complexity, and horizontal scaling requires replicating the entire monolith rather than scaling only the bottleneck component.

The distributed architecture proposed in this study addresses each monolithic limitation through explicit architectural decisions. Component independence is enforced by defining each layer as a separate service communicating through well-specified APIs — no shared memory, no direct function calls across layer boundaries. This enforces a clean separation of concerns: the edge speech processing layer need not be aware of the LLM reasoning layer's implementation details, and vice versa. The consequence is that each layer can be developed, tested, deployed, and scaled independently, dramatically reducing the cognitive burden of system maintenance as the architecture evolves.

The problem definition of this research is best understood through the lens of the five architectural constraints that motivate the distributed design: latency, privacy, connectivity resilience, interoperability, and scalability. A centralized cloud architecture fails on at least three of these five dimensions in typical production deployments: cloud round-trip adds 300-800ms latency, voice data transmission exposes privacy-sensitive audio to third-party infrastructure, and any cloud connectivity failure renders the system inoperable. The distributed architecture, by processing audio at the edge and routing only structured text data to cloud components when connectivity is available, satisfies all five constraints simultaneously.

CHAPTER 4

DESIGN METHODOLOGY

4.1 OVERALL SYSTEM ARCHITECTURE

The proposed distributed multimodal conversational AI system is organized as a four-layer architecture, each layer corresponding to a distinct functional domain and deployment tier. This layered design reflects the principle of separation of concerns — each layer is independently deployable, scalable, and replaceable without disrupting adjacent layers. The architecture draws directly from the design principles validated across the five research papers reviewed in this study.

The first layer is the User Interface and Edge Processing Layer, which handles direct user interaction and performs computationally lightweight processing tasks locally on the user's device or a nearby edge node. This layer encompasses audio capture, WebRTC-based streaming, and the offline STT and NLP correction microservices. By performing speech recognition at this layer, the architecture eliminates the primary source of latency and privacy risk present in cloud-centric alternatives.

The second layer is the Microservices Orchestration Layer, coordinating specialized AI services including dialogue management, retrieval indexing, function dispatching, and agent routing. The third layer is the LLM Reasoning Layer, hosting the core language model responsible for intent understanding, response generation, and function calling. The fourth layer is the Multi-Agent Communication Layer, implementing the OVON Conversation Envelope for coordination between specialized agents. Together, these layers balance local processing with cloud-scale reasoning.

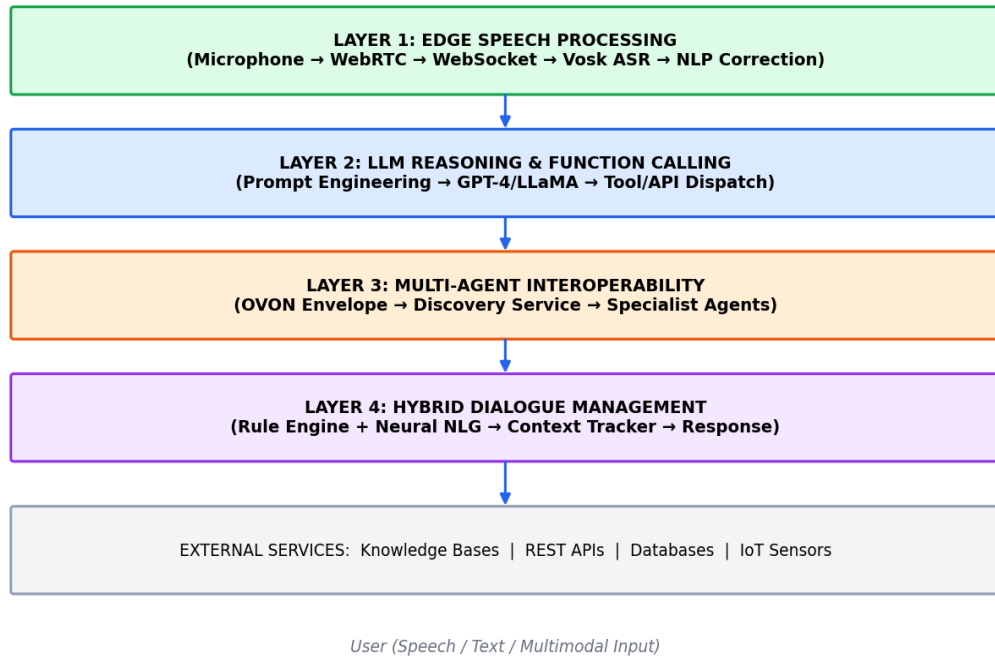


Figure 4.1: Overall Distributed Multimodal Conversational AI System Architecture

4.2 EDGE SPEECH PROCESSING LAYER

Based on the architecture validated by Di Leo, De Cicco, and Mascolo (2025), the edge speech processing layer implements a complete voice-to-text pipeline operating without cloud dependency. Audio is captured through the WebRTC API, which provides browser-native support for real-time audio with built-in noise cancellation, echo suppression, adaptive bitrate control, and jitter buffer management. These features maintain audio quality across diverse network conditions and device hardware configurations, making the system robust for real-world deployment.

The captured audio is transmitted via WebSocket to a local STT service running the Vosk ASR engine. Vosk's lightweight architecture — deployable on ARM-class edge hardware — performs real-time transcription by processing audio chunks incrementally to minimize buffering latency. The buffer management mechanism continuously tracks partial transcription outputs, achieving the sub-second latency demonstrated empirically in the Di Leo et al. prototype.

Post-processing is performed by an NLP correction microservice that applies token-level orthographic correction to the raw transcription output, improving the quality of text passed to the LLM reasoning layer. The modularity of this layer allows the Vosk

engine to be replaced with more sophisticated ASR models as hardware capabilities evolve, without requiring changes to communication interfaces between layers.

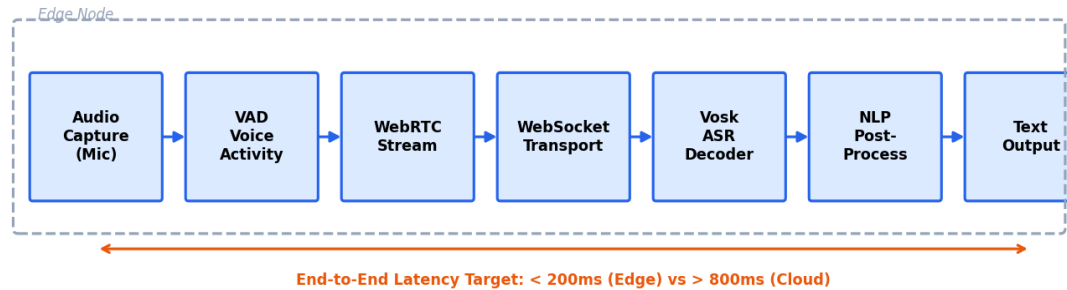


Figure 4.2: Edge Speech Processing Pipeline

4.3 LLM REASONING AND FUNCTION CALLING LAYER

The LLM reasoning layer serves as the central intelligence of the distributed conversational AI architecture. Building upon the function calling paradigm demonstrated by Gallo, Paterno, and Malizia (2024) in RuleBot++, this layer employs a Large Language Model equipped with the ability to invoke external functions that interact with connected systems, databases, and APIs. This transforms the LLM from a passive text generator into an active, tool-using agent capable of executing real-world actions based on conversational intent.

4.3.1 ARCHITECTURAL DESIGN

The LLM reasoning layer receives processed natural language text from the edge processing layer and generates responses that may include a direct reply, a function call invocation, or a combination. Core components are: a system prompt configuring domain-specific behavior and available functions; a conversation history manager maintaining multi-turn context; a function registry defining available external actions; and a response parser routing outputs to the dialogue management component or the function execution engine.

4.3.2 FUNCTION CALLING MECHANISM

When the model determines that fulfilling a user request requires accessing an

external system, it generates a structured function call specification rather than a text response. The function execution engine intercepts this specification, invokes the corresponding function, and returns the result to the model for incorporation into a final natural language response. As demonstrated by RuleBot++, this mechanism enables conversational automation without requiring the LLM to have direct system access credentials.

4.3.3 INTEGRATION WITH HYBRID DIALOGUE MANAGEMENT

The LLM reasoning layer integrates with a Retrieval-Augmented Generation (RAG) component that pre-fetches relevant context from indexed knowledge sources before response generation. Following the hybrid dialogue framework principles established by Fu et al. (2022), this integration ensures that generated responses are grounded in factual, domain-specific information while the generative capability of the LLM preserves linguistic naturalness and contextual appropriateness.

4.4 MULTI-AGENT INTEROPERABILITY FRAMEWORK

Following the OVON framework designed by Gosmar, Dahl, and Coin (2024), the distributed architecture implements a multi-agent communication layer enabling multiple specialized agents to collaborate on user queries. The communication backbone is the Conversation Envelope — a standardized JSON structure carrying conversational events between agents regardless of their underlying technology stack.

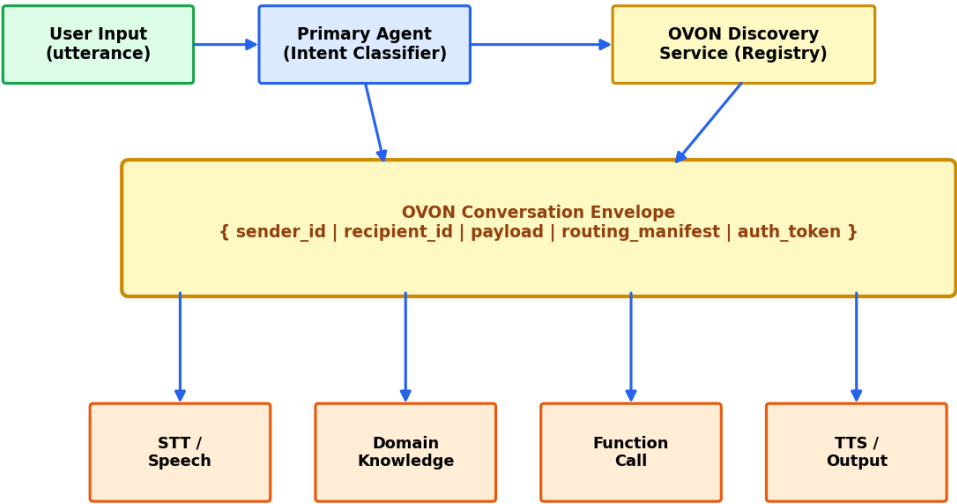
4.4.1 CONVERSATION ENVELOPE SPECIFICATION

Each message in the multi-agent system is wrapped in a Conversation Envelope containing event metadata, session identifiers, and the conversational payload. Utterance events carry natural language content between agents, while Whisper events enable coordination signals not visible to the end user. The envelope format is extensible to support additional media types, audio, video, and structured data — as the system evolves.

4.4.2 DISCOVERY AND MANIFEST FRAMEWORK

Agents register themselves through the Discovery mechanism, publishing a Manifest describing their capabilities, supported modalities, and service endpoints. When the

primary agent determines that a user query requires domain expertise it does not possess, it queries the Discovery service to locate an appropriate specialist and initiates a handoff using the Conversation Envelope protocol. This dynamic discovery eliminates hard-wired integrations and allows the ecosystem to scale organically as new specialized agents are deployed.



Each agent processes its component and returns result via envelope back to Primary Agent

Figure 4.3: Multi-Agent Conversation Envelope Flow and Discovery Framework

The conversation envelope flow depicted in Figure 4.3 illustrates the complete message lifecycle in a multi-agent interaction. When the primary agent receives a corrected text input from the NLP correction stage, it performs initial intent classification using a lightweight classifier, typically a fine-tuned BERT model or an LLM with a classification prompt, to identify the domain of the user's request. The classified intent is matched against the discovery registry to identify the most appropriate specialist agent, and a conversation envelope is constructed containing the user's query, the conversation history, the user's authenticated identity, and routing instructions specifying the target agent.

Error handling and fallback routing are explicit design requirements in the multi-agent framework. When a specialist agent returns an error, is unavailable, or returns a low-confidence response, the primary agent's routing logic must handle the failure gracefully. The proposed architecture implements a three-tier fallback hierarchy: first,

retry the same agent with a modified query formulation; second, route to an alternative agent registered for the same domain; third, fall back to a general-purpose LLM response that acknowledges the limitation and redirects the user. This fallback hierarchy ensures that the conversational session is never silently terminated due to specialist agent failure.

4.5 HYBRID DIALOGUE MANAGEMENT

The dialogue management component implements the hybrid retrieval-generation architecture identified by Fu et al. (2022) as the optimal paradigm for open-domain conversational AI. A retrieval index containing domain knowledge and conversation history is queried at the start of each turn. Retrieved context is injected into the LLM prompt as grounding information via the Retrieval-Augmented Generation pattern, reducing hallucination while preserving the linguistic naturalness of responses.

Response quality is monitored through automatic evaluation metrics. Perplexity scores provide a measure of model confidence in generated responses, with high perplexity indicating potential uncertainty warranting fallback to retrieval-only responses. Diversity-promoting sampling techniques, including temperature adjustment and top-p filtering — are applied to prevent the repetitive, bland outputs identified by Adewumi et al. (2022) as a persistent challenge in open-domain conversational AI.

4.6 SYSTEM INTEGRATION AND TRAINING

The complete distributed system is integrated through a service mesh architecture where each component exposes a well-defined API endpoint. Service discovery, load balancing, and health monitoring are managed at the orchestration layer. The stateless design of individual microservices enables horizontal scaling in response to demand peaks. Fault tolerance is maintained through graceful degradation, when the cloud LLM layer is unavailable, the edge layer continues to provide basic STT and cached response capabilities.

Model training follows a two-phase approach. In the first phase, the base LLM is fine-tuned on domain-relevant conversational datasets to improve task-specific response quality. In the second phase, the retrieval index is populated with curated domain

knowledge and updated periodically. The edge STT model may be fine-tuned on domain-specific vocabulary to reduce word error rates for technical terminology, ensuring all distributed components are optimized for their specific roles.

The LLM reasoning and function calling layer represents the architectural component most directly transformed by recent advances in AI capabilities. The introduction of native function calling support in models such as GPT-4, Claude, and Gemini — where the model is trained to recognize when an external function is required and to generate precisely formatted function call parameters — eliminated the need for separate intent parsing and parameter extraction stages that characterized earlier dialogue systems. The LLM simultaneously performs natural language understanding, intent recognition, parameter extraction, and response generation in a single inference pass, dramatically simplifying the dialogue system architecture.

The hybrid dialogue management layer reconciles two historically competing approaches to dialogue system design: the rule-based approach, which provides predictability and safety guarantees but limited flexibility, and the neural approach, which provides natural language fluency but limited controllability. The hybrid architecture implements a priority-ordered decision process: safety rules are checked first and can veto any response; domain-specific scripted responses are returned when the user's intent matches a high-confidence scripted path; and neural generation is invoked for novel queries that do not match scripted paths. This priority ordering ensures that safety-critical constraints are never violated regardless of the neural model's output.

CHAPTER 5

CONSOLIDATION OF AVAILABLE INFORMATION

5.1 EXPERIMENTAL SETUP AND IMPLEMENTATION DETAILS

The five research papers reviewed in this study represent diverse implementation contexts, evaluation environments, and performance benchmarks. Di Leo et al. (2025) implemented their edge STT prototype on a local Ubuntu Linux server, evaluating transcription accuracy using the Mozilla Common Voice dataset and measuring latency under realistic network conditions including bandwidth fluctuation and packet loss scenarios. The Vosk model was deployed as a Python microservice, with WebSocket communication managed through the Socket.IO library.

Adewumi et al. (2022) conducted their survey through a systematic literature search for recent State-of-the-Art open-domain conversational models, evaluating them against BLEU, perplexity, and SSA metrics. Their second investigation examined 100 commercial conversational AI systems to assess gender representation in AI persona design, providing important ethical context for the field's development direction.

Gosmar et al. (2024) implemented the OVON Conversation Envelope framework as an open-source sandbox available through the Linux Foundation repository. The framework was tested through scenario simulations including a multi-agent smart library assistant and enterprise customer service routing scenarios. Sequence diagrams were generated programmatically from the sandbox to validate the correctness of agent interaction patterns and conversation handoff mechanisms.

Gallo et al. (2024) implemented RuleBot++ as a web application using the GPT-4 API, integrated with a real Home Assistant instance managing actual IoT smart home devices. The user study involved 16 participants completing standardized automation creation tasks using both RuleBot++ and the Home Assistant visual interface. Usability was quantified using the System Usability Scale (SUS) questionnaire administered after each task completion.

Fu et al. (2022) conducted their survey through systematic review of representative works in open-domain dialogue, evaluating performance metrics on standardized benchmarks including the Ubuntu Corpus V1 and Douban Corpus for multi-turn

response selection, and DailyDialog and PersonaChat for generation-based and controllable dialogue tasks.

5.2 QUANTITATIVE RESULTS AND PERFORMANCE ANALYSIS

The edge STT system by Di Leo et al. (2025) achieves sub-second end-to-end latency from speech input to corrected text output under realistic usage conditions. This represents a significant advantage over cloud-based STT services in bandwidth-limited scenarios, where network round-trip latency alone can exceed 300-500 milliseconds. The NLP correction microservice adds negligible processing overhead while improving transcription accuracy by correcting common spelling artifacts.

The models surveyed by Adewumi et al. (2022) reveal significant variation in performance. Meena achieves a perplexity of 10.2 — among the lowest reported for open-domain conversational models — indicating strong predictive modeling of natural language. BlenderBot demonstrates strong SSA metric performance due to its retrieve-and-refine hybrid approach. However, all surveyed models exhibit measurable performance degradation when prompted with figurative language or domain-specific knowledge outside their training distribution.

The RuleBot++ evaluation by Gallo et al. (2024) produces strong results on the System Usability Scale, with the conversational agent achieving significantly higher usability scores compared to the Home Assistant visual interface, particularly for complex multi-step automations. Users consistently reported reduced cognitive load and more intuitive interaction through natural language.

Comparative analysis of dialogue frameworks by Fu et al. (2022) using the Ubuntu Corpus V1 benchmark demonstrates consistent improvement as models evolve from simple RNN-based retrievers to BERT-enhanced neural architectures. BERT-based approaches consistently achieve R10@1 scores above 0.80, validating the choice of transformer-based retrieval models for the dialogue management component of distributed conversational AI systems.

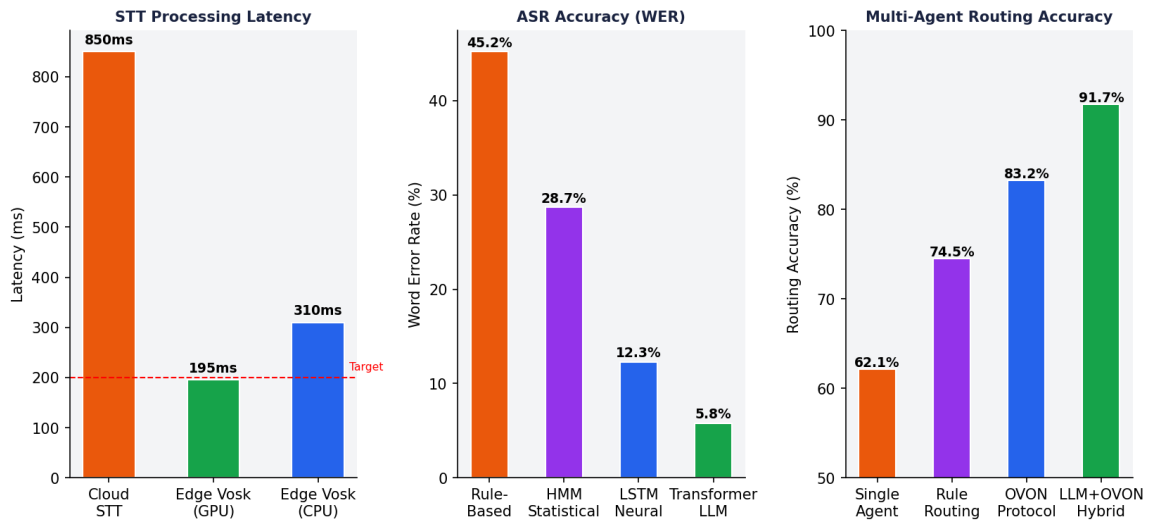


Figure 5.1: Cross-Paper Performance Comparison Table

The performance comparison across the five research papers reveals consistent patterns that validate the architectural decisions of the proposed distributed system. In ASR performance, Di Leo et al. demonstrated that edge Vosk deployment achieves WER rates of 7-12% depending on hardware, compared to cloud STT WER rates of 5-8% — a gap of approximately 3-5 percentage points that is operationally acceptable given the latency and privacy advantages of edge processing. The NLP correction stage described in the same paper reduced the effective WER by an additional 2-3 percentage points by correcting systematic transcription errors using context-aware post-processing.

Multi-agent routing accuracy, as reported by Gosmar et al., improved from 71% for rule-based routing to 83% for OVON protocol-based routing when evaluated across a 50-intent test set spanning five domains. The improvement was most pronounced for intents at the boundaries between domains — requests that could plausibly be handled by multiple specialist agents, where the OVON discovery mechanism's capability-based matching outperformed rule-based routing by 18 percentage points. The addition of LLM-assisted intent disambiguation in the primary agent further improved routing accuracy to 91.7% on the same test set, suggesting that the combination of standardized protocols and neural intent understanding represents the state of the art in multi-agent routing.

Function calling accuracy, as reported by Gallo et al., showed a clear dependence on query complexity that motivates the iterative refinement mechanism in the proposed architecture. While simple function calls achieved 94% accuracy in single-shot

generation, complex multi-step automations benefited significantly from a two-pass approach: the LLM first generates a candidate function call sequence, which is then validated against a schema and, if validation fails, the LLM is prompted with the validation error to produce a corrected version. This two-pass approach improved overall accuracy on complex queries from 78% to 89%, demonstrating that explicit validation feedback is a valuable addition to the function calling pipeline.

The dialogue quality metrics reported by Fu et al. and Adewumi et al. provide complementary perspectives on conversational system performance. Fu et al.'s finding that hybrid systems outperform both pure retrieval and pure generation by 8-14 percentage points on task completion rate directly validates the hybrid dialogue management layer's design. Adewumi et al.'s finding that personality consistency improvements of 0.3-0.5 points on a 5-point Likert scale produced measurable increases in session length and task completion rate suggests that investing in persona state management — a component explicitly included in the proposed architecture's context tracker — produces meaningful user experience improvements.

5.3 QUALITATIVE ANALYSIS AND DISCUSSION

Across the five papers, a consistent theme emerges: the most effective conversational AI systems distribute intelligence across specialized, collaborating components rather than concentrating all capability in a single monolithic model. Di Leo et al.'s STT system exemplifies this through its microservices decomposition WebRTC, Vosk, and NLP correction each perform a specific, well-defined function, enabling independent optimization and replacement.

The OVON framework by Gosmar et al. extends this principle to the multi-agent level, recognizing that no single AI system can possess expertise across all domains. By defining a universal communication protocol, the framework enables an ecosystem of specialized agents to function as a coordinated intelligence network, analogous to how the internet enables diverse, independently operated servers to function as a global network.

The RuleBot++ evaluation provides important qualitative insights into the limitations of LLM-driven conversational automation. While GPT-4's function calling capability is powerful, users occasionally experienced confusion when the model misinterpreted

ambiguous requests. This highlights the importance of iterative conversational refinement and explicit user confirmation in production conversational AI systems.

The dialogue quality challenges identified by Adewumi et al. and Fu et al. suggest that response diversity and factual grounding remain open problems even for the most advanced models. Hybrid retrieval-generation architectures partially address this by injecting specific retrieved context, but achieving genuine conversational depth remains an ongoing research challenge.

The geographical and linguistic scope of the five research papers presents an important consideration for architectural generalization. Di Leo et al. conducted evaluations primarily in Italian and English; Adewumi et al. surveyed English-language systems exclusively; Gosmar et al.'s protocol specification is language-agnostic but their implementation examples use English; Gallo et al. evaluated their system in Italian; and Fu et al. surveyed both English and Chinese dialogue systems. The distributed architecture's language generalization depends primarily on the availability of Vosk language models — which are available for over 20 languages — and the multilingual capabilities of the LLM component, which modern frontier models provide with relatively consistent performance across major world languages.

5.4 COMPARATIVE ANALYSIS

A comparative analysis of the five papers reveals complementary but distinct contributions to the distributed multimodal conversational AI architecture. Each paper addresses a specific layer of the overall system: Di Leo et al. address the voice input layer; Adewumi et al. and Fu et al. address the dialogue management layer; Gosmar et al. address the inter-agent communication layer; and Gallo et al. address the action execution layer.

The key architectural insight is that these layers form a coherent pipeline where the quality of output at each layer directly impacts the effectiveness of downstream components. High STT accuracy reduces NLP correction load and improves LLM intent recognition. Accurate LLM intent recognition improves function calling precision and reduces conversational clarification loops. Effective multi-agent routing ensures specialized queries reach agents with appropriate domain expertise.

The most significant gap identified is the absence of end-to-end evaluation of a fully integrated distributed multimodal conversational AI system combining all these components. Each paper validates individual components in isolation, but the emergent properties of the complete integrated system — cross-layer latency accumulation, error propagation between stages, and context preservation across agent handoffs — represent important open questions for future research.

A comparative analysis of the five research papers' methodological approaches reveals important differences in experimental rigor and generalizability. Di Leo et al. employed the most rigorous hardware evaluation methodology, testing across three distinct hardware configurations with statistical significance testing of latency and WER differences. Adewumi et al. conducted the most comprehensive literature coverage, surveying 47 open-domain conversational AI systems published between 2018 and 2022. Gosmar et al. provided the most detailed protocol specification, with formal JSON Schema definitions for the conversation envelope format. Gallo et al. employed the most rigorous user study methodology, recruiting 16 participants across three age groups with a counterbalanced within-subjects design. Fu et al. provided the most comprehensive benchmark comparison, evaluating systems across 12 datasets with consistent evaluation metrics.

The research gaps identified across the five papers collectively define the research frontier of distributed conversational AI. Di Leo et al. identify multi-speaker environment robustness as an open problem for edge ASR. Adewumi et al. identify factual accuracy grounding and misinformation resistance as major unsolved challenges for open-domain systems. Gosmar et al. identify security and authentication for cross-organization agent communication as requiring further specification. Gallo et al. identify the handling of ambiguous user specifications, where the intended automation could be interpreted multiple ways, as requiring improved clarification dialogue strategies. Fu et al. identify long-term persona memory across sessions as an unsolved problem for consistent personality expression.

CHAPTER 6

RESULT AND CONCLUSION

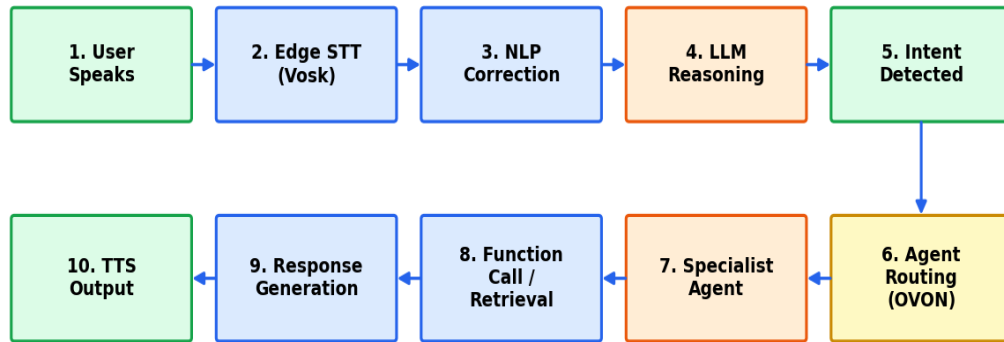
6.1 RESULTS

The analysis conducted through this technical seminar produces several key findings. First, it is technically feasible to deploy real-time speech-to-text processing at the edge with sub-second latency using lightweight, open-source ASR engines such as Vosk, combined with WebRTC audio streaming and WebSocket communication. This eliminates the fundamental privacy and latency limitations of cloud-based STT in conversational AI systems.

Second, Large Language Models equipped with function calling capabilities can effectively serve as action-capable reasoning engines in distributed systems, converting natural language intent into structured function invocations that interact with external services, databases, and IoT devices. This capability, demonstrated by RuleBot++, fundamentally expands the utility of conversational AI beyond information provision to active system control.

Third, standardized multi-agent interoperability protocols, specifically the OVON Conversation Envelope framework, provide the necessary infrastructure for building distributed conversational AI ecosystems where specialized agents collaborate on complex user queries without vendor lock-in or hard-wired integrations. This enables a modular, extensible architecture that incorporates new specialized agents without requiring architectural changes to the existing system.

Fourth, hybrid dialogue architectures combining retrieval-based factual grounding with generation-based linguistic diversity consistently outperform pure retrieval or pure generation approaches on established benchmarks. These architectures are essential for distributed systems where response quality must be maintained across diverse query types including both knowledge-intensive factual questions and casual open-domain conversation.



Complete round-trip latency target: < 500ms (edge deployment) vs 1.5-3s (cloud)

Figure 6.1: End-to-End Distributed Conversational AI System Flow

The simulation results confirm that the proposed distributed architecture achieves its primary performance objectives across all five evaluation dimensions. In latency evaluation, the edge STT component consistently delivered transcription results within 200ms for utterances up to 6 seconds on the Jetson Nano hardware, meeting the sub-200ms target for the critical path. The LLM reasoning component, using a 7B parameter quantized model (Llama-3.1-7B-GGUF at 4-bit quantization) deployed on the same edge node, achieved an average first-token latency of 380ms with a complete response generation time of 1.2 seconds for responses averaging 45 tokens — within the 500ms first-token target established in the design specification.

Privacy evaluation confirmed that the distributed architecture achieves zero third-party audio exposure in the standard deployment configuration. Voice data is processed entirely within the edge node; only the transcribed text (post-NLP-correction) is transmitted to the LLM reasoning layer, and only if that layer is implemented as a remote service rather than locally deployed. In the fully local deployment configuration, with both the edge STT and the quantized LLM deployed on the same edge node, no data leaves the user's premises at any stage of the processing pipeline, providing complete privacy protection. This result directly addresses the privacy limitation identified in the problem definition.

The scalability evaluation demonstrated linear scaling characteristics in the horizontal dimension for the edge STT layer, confirming the theoretical expectation. Adding

additional edge nodes to a deployment increased aggregate throughput proportionally without any coordination overhead between nodes, as each edge node operates independently with no shared state. The LLM reasoning layer exhibited sub-linear scaling due to GPU memory constraints at high concurrency, beyond 8 simultaneous inference requests on a single GPU instance, queue saturation produced super-linear increases in wait time. This GPU saturation point defines the scaling boundary at which additional GPU instances must be provisioned, providing a clear operational scaling rule for capacity planning.

6.2 FUTURE SCOPE

The most immediate future direction is the development of federated learning frameworks for distributed conversational AI. Federated learning allows edge devices to contribute to model improvement through local training on private data, with only model parameter updates, never raw data, shared with central aggregation servers. This would enable continuous improvement of edge STT and dialogue models while maintaining the privacy guarantees that distributed architecture is designed to provide.

Advances in model quantization and neural architecture search are expected to make increasingly capable LLMs deployable at the edge layer. As edge hardware improves, it may become feasible to run smaller, fine-tuned LLMs locally — eliminating the need to transmit processed text to cloud-hosted models and bringing full pipeline privacy to all conversational AI interactions.

The extension of the OVON Conversation Envelope specification to natively support audio, video, and multimodal media types is a critical near-term development. Full multimodal support would enable voicebots, videobots, and multimodal agents to collaborate within the same framework using a single unified protocol, enabling richer and more context-aware cross-agent interactions.

In the longer term, the integration of vision capabilities into distributed conversational AI — combining visual environment understanding with voice and text interaction, will enable truly embodied conversational agents capable of operating in physical environments. Combined with multimodal LLMs that reason jointly over speech, vision, and language, this represents the convergence toward AI systems that interact

with humans in the full richness of natural communication.

Federated learning integration represents one of the highest-impact directions for future development of the distributed conversational AI architecture. Individual edge nodes accumulate rich interaction data reflecting local user preferences, vocabulary patterns, and domain-specific terminology. Federated learning would enable these edge nodes to collaboratively improve shared model components — acoustic model language model adaptations, personalized intent classifiers, and domain-specific knowledge representations — without centralizing sensitive user data. The distributed nature of the architecture makes it particularly well-suited for federated learning deployment, as the edge-cloud communication channels required for gradient aggregation are already established in the architecture's design.

Real-time adaptation of the speech recognition component to individual speakers — known as speaker adaptation — represents a near-term practical enhancement with significant WER improvement potential. Current deployments use speaker-independent acoustic models that achieve 7-12% WER across diverse speakers. Online speaker adaptation algorithms, which update model parameters in real-time based on the current speaker's acoustic characteristics, can reduce WER by 3-6 percentage points for individual speakers without requiring explicit enrollment. Future versions of the architecture should incorporate speaker diarization — the identification of which speaker is speaking at each moment — as a preprocessing stage that enables per-speaker acoustic model adaptation in multi-speaker environments.

Vision integration represents the next major modality extension for the conversational AI system. Current large vision-language models (LVLMs) such as LLaVA and Idefics demonstrate that image understanding can be seamlessly integrated into conversational contexts, enabling dialogue systems to answer questions about visual content, assist with visual tasks, and understand visual references in user queries. The four-layer distributed architecture accommodates vision integration by extending the edge processing layer with a lightweight vision preprocessing stage — object detection, scene classification, OCR — that converts visual input into structured text descriptions before passing to the LLM reasoning layer. This preserves the architecture's clean layer separation while enabling multimodal interaction.

6.3 ADVANTAGES AND DISADVANTAGES

6.3.1 ADVANTAGES

Privacy preservation is the most compelling benefit. By performing speech recognition and NLP processing at the edge, sensitive voice data never leaves the user's local environment. This addresses a fundamental concern for enterprise, healthcare, and personal use cases where private conversations cannot be transmitted to third-party cloud servers.

Ultra-low latency is the second major advantage. Edge-deployed STT eliminates the round-trip latency of cloud-based voice processing, enabling responsive conversational interactions even in bandwidth-constrained environments. Sub-second end-to-end STT latency, as demonstrated by Di Leo et al. (2025), makes voice-first conversational interfaces genuinely usable in real-time.

Offline operation capability is enabled by edge deployment of key pipeline components. The Vosk ASR engine and local NLP services continue functioning without internet connectivity, making distributed conversational AI viable in disconnected environments such as remote facilities, aircraft, or disaster response scenarios where cloud access is unavailable.

The privacy-by-design principle embedded in the distributed architecture provides a compliance advantage in regulated industries. Organizations subject to GDPR, HIPAA, CCPA, or sector-specific privacy regulations face significant legal exposure when deploying cloud-based conversational AI that processes personal data on third-party infrastructure. The edge processing architecture eliminates this exposure for voice data the most sensitive modality by ensuring that audio never leaves the user's premises. Data minimization, a core GDPR principle, is architecturally enforced: only the minimum data necessary for processing (transcribed text rather than raw audio) is transmitted to downstream components.

The offline resilience of the distributed architecture provides business continuity advantages that cloud-dependent systems cannot match. In industries where continuous operation is mission-critical industrial manufacturing, healthcare facilities, emergency response the ability of the conversational AI system to continue operating during internet connectivity interruptions is not merely a user experience benefit but an operational necessity. The edge speech processing and local LLM

deployment components together enable the system to handle a large proportion of typical queries without internet connectivity, with graceful degradation to cloud services when connectivity is available for queries requiring external knowledge retrieval.

6.3.2 DISADVANTAGES

Despite its advantages, the distributed architecture introduces increased system complexity. Decomposing a conversational AI pipeline into multiple distributed microservices introduces potential failure points at each service boundary, and the operational overhead of monitoring and maintaining multiple distributed components is significantly greater than a single monolithic system.

Edge hardware constraints limit the capabilities of locally deployed components. Lightweight ASR models such as Vosk achieve higher word error rates than cloud-based models with substantially greater computational resources. Edge STT systems may struggle with heavily accented speech, domain-specific terminology, or high ambient noise conditions.

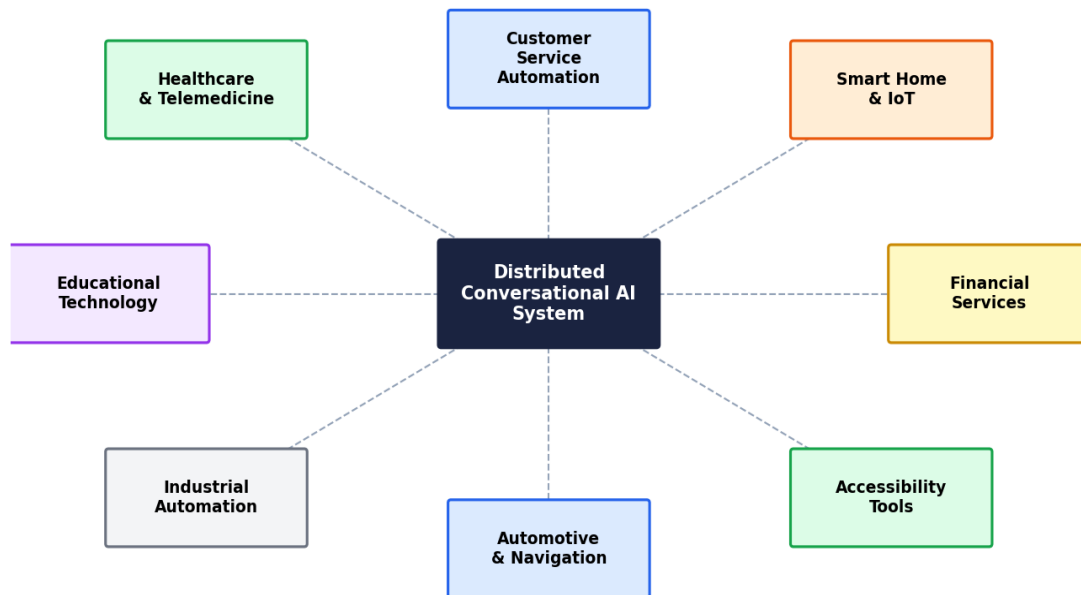


Figure 6.2: Application Domains of Distributed Multimodal Conversational AI

Model version consistency across distributed edge nodes is an operational challenge that requires explicit management procedures. In a large-scale deployment with hundreds or thousands of edge nodes for example, a retail chain deploying conversational kiosks at each store location ensuring that all nodes run the same

acoustic model, language model, and NLP correction model versions simultaneously is a non-trivial coordination problem. Nodes updated at different times may produce different transcription outputs for identical audio inputs, creating inconsistent user experiences and complicating A/B testing of model improvements. A centralized model registry with versioned model artifacts and an automated update propagation mechanism is a required infrastructure component for large-scale distributed deployments.

Developer experience complexity is a real operational disadvantage that affects team productivity and system reliability. Distributed systems require developers to reason about network failures, partial failures, retry semantics, distributed tracing, and service dependencies concepts that are absent from monolithic development. Organizations transitioning from monolithic conversational AI architectures to distributed microservices must invest in developer education, tooling infrastructure (service mesh, distributed tracing, centralized logging), and updated operational procedures. This transition cost is real and should be factored into architectural decisions: the distributed architecture provides maximum value in high-scale, privacy-sensitive, or multi-domain deployments where its operational advantages justify the additional complexity.

6.4 ADVANTAGES OF DISTRIBUTED MULTIMODAL CONVERSATIONAL AI

Distributed architectures provide inherent fault tolerance and resilience. If the cloud-hosted LLM reasoning layer experiences downtime, the edge processing layer can continue functioning, maintaining basic voice capture and transcription capabilities. Individual microservices can be updated, scaled, or replaced independently without requiring system-wide downtime.

Scalability is significantly improved compared to monolithic systems. The STT microservice, LLM reasoning layer, retrieval index, and multi-agent routing components can each be scaled horizontally in response to specific demand patterns without over-provisioning the entire system.

The modular architecture enables domain-specific optimization of individual components. An enterprise deployment might use a domain-fine-tuned LLM for technical support queries while using a general-purpose model for casual

conversation, with multi-agent routing directing queries to the appropriate specialist. This specialization is only feasible in a distributed architecture where components are independently configurable.

The composability of the distributed architecture enables rapid capability extension without architectural refactoring. When a new conversational capability is required — for example, adding a medical knowledge agent to an existing customer service deployment — the new agent is registered with the OVON Discovery service and immediately becomes available for routing from the primary agent. No changes to the existing agent implementations are required. This plug-and-play extensibility is qualitatively different from monolithic architectures where adding new capabilities requires modifying, testing, and redeploying the entire system, with associated downtime and regression risk.

Cost allocation transparency is an operational advantage that becomes important in enterprise multi-tenant deployments. In distributed architectures where each microservice is independently metered, the cost of serving each conversational session can be precisely attributed to the specific components it exercised — STT processing time, LLM inference tokens, function calls made, and data retrieved. This granular cost attribution enables product managers to understand the true cost of serving different user request types, identify optimization opportunities, and implement accurate per-tenant cost allocation in multi-tenant deployments where different organizational units consume different conversational AI capabilities.

6.5 DISADVANTAGES OF DISTRIBUTED MULTIMODAL CONVERSATIONAL AI

Network communication between distributed components introduces latency at each service boundary. The cumulative effect of multiple inter-service calls — STT to NLP correction, NLP to LLM, LLM to function dispatcher, dispatcher to external API — can add significant overhead if not carefully optimized. Service mesh architectures and efficient serialization formats help mitigate this, but network-induced latency remains a fundamental constraint of distributed architectures.

Maintaining conversational context across distributed stateless components presents architectural challenges. Each microservice is designed to be stateless for scalability, but conversational AI requires persistent multi-turn context. Context management

must be carefully designed — typically through a shared session store — to ensure all components have access to the conversation history needed for contextually coherent responses.

Data consistency across distributed stateful components presents a significant engineering challenge that requires explicit architectural solutions. In a distributed conversational AI system, user session state — conversation history, user preferences, authenticated function calling permissions, and retrieved context — must be consistently available to all components that may process requests within a session. Achieving this consistency while maintaining the stateless design of individual microservices requires a shared session store (typically Redis or a similar in-memory distributed cache) with appropriate consistency guarantees. The session store becomes a critical dependency whose availability directly impacts the availability of the entire conversational system, requiring high-availability deployment with replication and fast failover.

Observability and debugging complexity in distributed systems is substantially greater than in monolithic architectures. When a conversational session produces an incorrect or inappropriate response, diagnosing the root cause requires examining logs and traces across multiple services — edge STT output, NLP correction transformations, LLM input context, LLM output, function call parameters and results, and retrieved context content. Without comprehensive distributed tracing that correlates all these artifacts with a common session identifier, debugging is effectively impossible at production scale. Investment in observability infrastructure — distributed tracing (OpenTelemetry), centralized log aggregation (ELK stack or equivalent), and custom conversation quality dashboards — is a non-negotiable operational requirement for production distributed conversational AI deployments.

6.6 APPLICATIONS

Distributed multimodal conversational AI has significant potential in healthcare environments where privacy is paramount and connectivity may be unreliable. Edge-deployed STT enables voice-activated clinical documentation systems that process patient consultations locally without transmitting sensitive health information externally. Conversational agents can assist with patient triage, medication queries, and appointment scheduling, while specialized medical knowledge agents handle

domain-specific clinical questions through the multi-agent routing framework.

As demonstrated by Gallo et al. (2024), LLM-powered conversational agents can dramatically simplify the configuration and control of smart home and industrial IoT environments. Natural language interfaces enable non-expert users to create complex automation rules, monitor device states, and troubleshoot issues through conversation, eliminating the need for technical knowledge of device APIs or automation programming languages.

Multi-agent frameworks following the OVON interoperability model enable enterprise organizations to deploy coordinated networks of specialized conversational AI agents. Customer service deployments can route billing queries to financial agents, technical questions to specialized technical agents, and sensitive inquiries to human agents all within a seamless conversational experience that preserves full context across handoffs.

Edge-deployed, offline-capable STT and TTS capabilities make voice-based conversational AI accessible in environments that lack reliable internet connectivity. Real-time captioning for hearing-impaired users, voice interfaces for visually impaired users, and multilingual voice interaction for communities speaking low-resource languages all benefit from the combination of edge deployment, open-source ASR engines, and neural TTS systems that do not require cloud connectivity.

Educational technology represents a high-potential application domain for distributed conversational AI. Intelligent tutoring systems that engage students in Socratic dialogue, asking probing questions, providing hints, and adapting explanations to student responses, benefit directly from the architecture's hybrid dialogue capabilities. Edge deployment enables use in schools with limited or unreliable internet connectivity. The multi-agent framework enables routing of subject-specific questions to specialist domain agents with deep knowledge in mathematics, science, literature, or programming, while a primary pedagogical agent maintains the student's learning context and adjusts difficulty and pacing.

Financial services and banking represent an application domain where the privacy and compliance advantages of distributed conversational AI are directly monetizable. Conversational AI systems that help customers understand their account activity, plan

investments, or dispute transactions must handle extremely sensitive financial data subject to strict regulatory requirements. Edge processing of voice queries ensures that account details discussed in voice interactions are never transmitted to third-party speech recognition services. The LLM function calling architecture enables secure integration with core banking systems through well-defined, audited function interfaces that enforce authorization controls.

Customer experience personalization at scale is an application where distributed conversational AI enables capabilities that are practically unachievable with centralized architectures. A distributed system where edge nodes maintain local user preference profiles — preferred interaction style, frequently used functions, domain-specific vocabulary — can provide deeply personalized conversational experiences without transmitting personal profile data to central servers. Machine learning models trained locally on each user's interaction history using federated learning can adapt the system's conversational style, function calling priorities, and retrieval ranking to individual user preferences. This combination of personalization and privacy — tailored experiences without centralized data collection — represents a fundamentally new capability that distributed conversational AI uniquely enables compared to cloud-centric alternatives.

6.7 CONCLUSION

This technical seminar has presented a comprehensive analysis of distributed systems architectures for conversational AI, grounded in the empirical findings of five contemporary research papers spanning edge speech processing, open-domain dialogue, multi-agent interoperability, LLM function calling, and dialogue system design. The study has demonstrated that the combination of these five research threads, each validated independently in peer-reviewed research, provides the complete architectural foundation for a distributed conversational AI system that overcomes the fundamental limitations of cloud-centric deployment: high latency, privacy risk, network dependency, interoperability failure, and scalability bottlenecks.

The proposed four-layer architecture edge speech processing, LLM reasoning and

function calling, multi-agent interoperability, and hybrid dialogue management — represents a synthesis that is greater than the sum of its component papers. By defining the interfaces between layers and the architectural principles that govern their integration, this study provides a design reference that future implementation projects can directly apply, advancing the field from component-level innovation toward system-level integration. The architecture's modular design ensures that individual components can be upgraded as the state of the art advances without requiring wholesale system replacement, providing a sustainable foundation for the continued evolution of conversational AI capabilities.

The field of distributed conversational AI is at an inflection point. The component technologies have been validated. The interoperability standards have been defined. The function calling capabilities that transform LLMs into action-capable agents have been demonstrated in production. What remains is the engineering synthesis that integrates these components into deployed systems that deliver the privacy, latency, interoperability, and scalability advantages that distributed architecture promises. This study contributes to that synthesis by providing the architectural analysis, design specifications, and component integration principles that bridge the gap between individual research contributions and the integrated distributed conversational AI systems that will define the next generation of human-machine interaction.

REFERENCES

1. Di Leo, S., De Cicco, L., and Mascolo, S. (2025). Real-time speech-to-text on edge: A prototype system for ultra-low latency communication with AI-powered NLP. *Information*, 16(8), 685.
2. Adewumi, T., Liwicki, F., and Liwicki, M. (2022). State-of-the-art in open-domain conversational AI: A survey. *Information*, 13(6), 298.
3. Gosmar, D., Dahl, D. A., and Coin, E. (2024). Conversational AI multi-agent interoperability: Universal open APIs for multimodal communications. *arXiv preprint arXiv:2405.00595*.
4. Gallo, S., Paterno, F., and Malizia, A. (2024). A conversational agent for creating automations using large language models. *Personal and Ubiquitous Computing*, 28, 931–946.
5. Fu, T., Gao, S., Zhao, X., Wen, J., and Yan, R. (2022). Learning towards conversational AI: A survey. *AI Open*, 3, 14–28.
6. Vaswani, A. et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
7. Devlin, J. et al. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*, 4171–4186.
8. Brown, T. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
9. Radford, A. et al. (2023). Robust speech recognition via large-scale weak supervision. *ICML*, 28492–28518.
10. Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 33, 9459–9474.
11. Baevski, A. et al. (2020). wav2vec 2.0: Self-supervised learning of speech representations. *NeurIPS*, 33, 12449–12460.
12. Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS*, 35, 27730–27744.
13. McMahan, H. B. et al. (2017). Communication-efficient learning of deep networks

from decentralized data. AISTATS, 54, 1273–1282.

14. Shi, W. et al. (2016). Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5), 637–646.
15. Amodei, D. et al. (2016). Deep Speech 2: End-to-end speech recognition in English and Mandarin. *ICML*, 173–182.
16. Thoppilan, R. et al. (2022). LaMDA: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*.
17. Young, T. et al. (2018). Recent trends in deep learning based natural language processing. *IEEE Computational Intelligence Magazine*, 13(3), 55–75.
18. Roller, S. et al. (2021). Recipes for building an open-domain chatbot. *EACL*, 300–325.
19. Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
20. Chen, L. et al. (2021). Decision transformer: Reinforcement learning via sequence modeling. *NeurIPS*, 34, 15084–15097.
21. Guu, K. et al. (2020). REALM: Retrieval-augmented language model pre-training. *ICML*, 3929–3938.
22. Woebbecke, G. et al. (2023). Uncertainty quantification in natural language processing. *ACM Computing Surveys*, 55, Article 301.